哈尔滨工业大学　法国里昂商学院
Harbin Institute of Technology　emlyon business school

Object Oriented Programming
Python BootCamp

黄子扬

Ziyang Huang

二叉苹果树

June 13, 2026

Preface / 前言

本课程 **Object Oriented Programming**（课程代码 22EM31202C）是哈尔滨工业大学与法国里昂商学院联合举办的**应用数据科学本科项目（BSc in Applied Data Science）**一年级春季学期专业核心课，共 32 学时、2 学分，考核方式为**考试课**。课程由 **Imène BRIGUI** 老师与 **Saeed VARASTEN YAZDI** 老师联合讲授。

课程从 Python 变量与数据类型起步，逐步深入控制结构、数据结构、模块与包、面向对象编程（类、对象、封装、继承、多态）、文件 IO 与异常处理，最终覆盖数据科学标准库 Matplotlib 和 Pandas 的实战应用，强调理论与实践并重。

关于作者：黄子扬（ID：二叉苹果树），哈尔滨工业大学经济与管理学院 2025 级大数据管理与应用（中外合作办学）本科生。这份笔记诞生于期末备考阶段——将 9 章课程 PDF 逐一拆解，用 LaTeX 重排为结构化双语笔记，确保覆盖原课件全部内容。初稿完成后，借助 Claude Code + DeepSeek V4 Pro 辅助完成了格式优化、标题统一、附录速查、封面排版等打磨工作，最终形成这份完整课程笔记。

相关资源与更新见 GitHub：`huangziyangggg/OOP-Course-Notes`

Contents

1 01 - Variables & Data Types / 变量与数据类型

1.1 Variables / 变量基础

【Concept / 核心概念】What is a Variable? / 什么是变量？

In Python, variables are names that can be assigned a value and then used to refer to that value throughout your code.
在 Python 中，变量是可以被赋值并在代码中反复引用的名称。
Variables are fundamental to programming for two reasons:

1. **Keep values accessible / 保持值可访问**: You can assign the result of a time-consuming operation to a variable so your program doesn't have to perform the operation each time you need the result.
2. **Give values context / 赋予值上下文**: The number 28 could mean lots of different things. Giving it a name like `num_students` makes the meaning clear.

1.1.1 The Assignment Operator / 赋值运算符

【Concept / 核心概念】How it works / 赋值逻辑

The `=` symbol is called the **assignment operator**. It takes the value on the **right** and assigns it to the name on the **left**.
Python 使用 `=` 符号进行赋值。它将等号右侧的值" 绑定" 到左侧的名字上。

【Example / 实战案例】Basic Assignment / 基础赋值

```
age = 25
print(age)      # Output: 25
```

【Concept / 核心概念】How print() Works / print() 的工作原理

`print()` is a function that displays output. Python looks for the variable name, finds its assigned value, and **replaces** the variable name with its value before calling the function.
`print()` 是一个函数，用于显示输出。Python 先查找变量名对应的值，**用值替换**变量名后再调用函数。

1.1.2 Naming Rules & Conventions / 命名规则与规范

- **Valid Characters / 合法字符**: Uppercase and lowercase letters (A–Z, a–z), digits (0–9), and underscores (_). Variable names can be as long or as short as you like.
- **Case Sensitivity / 大小写敏感**: `age` is not the same as `Age`. Using `print(Age)` when only `age` is defined will raise a `NameError`.
- **Style / 规范**: Common practice is to write variable names in `lower_case_with_underscores` (e.g., `car_length`).

【Warning / 避坑指南】Variable Name Rules / 变量命名禁忌

- **No Digit Start / 严禁数字开头**: Names cannot begin with a digit. `3text = 4` is invalid.
- **Reserved Words / 保留字**: Do NOT name variables `print`, `for`, `in`, `list`, `str`, `int` —these are built-in functions you will override.
- **Meaningful Names / 命名要有意义**: Always use names that describe the data (e.g., `car_length` instead of `L`).
- **Don't make names too long / 不要太长**: Find a balance between descriptiveness and length.

【Example / 实战案例】Valid and Invalid Names / 合法与非法命名

```
# Valid names / 合法命名
test1 = 1
_alpha = 2
L = 3
car_length = 2000

# Invalid names / 非法命名
# 3text = 4          # Error: cannot start with digit
# my-var = 5         # Error: hyphen is not allowed
```

1.1.3 Comments / 代码注释

【Concept / 核心概念】What are Comments? / 什么是注释？

Comments are lines of text that don't affect how a program runs. They document what code does or why the programmer made certain decisions.
注释是不影响程序运行的文本行，用于记录代码用途或编程决策的原因。
The most common way to write a comment is to begin a new line with the `#` character. When Python runs your code, it ignores lines starting with `#`.

【Example / 实战案例】Comment Examples / 注释示例

```
# This variable stores the car length in cm
L = 2000
```

```python
print(L)

H = 146   # This variable stores the car height in cm
print(H)
```

1.2 Data Types / 数据类型百科

【Concept / 核心概念】What is a Data Type? / 什么是数据类型？

The term **data type** refers to what kind of data a value represents. Python has several built-in data types.
数据类型指一个值代表什么类型的数据。Python 有多种内置数据类型。

1.2.1 Primitive Types / 原始类型

Type / 类型	Keyword	Description / 描述	Example / 示例
Integer / 整数	`int`	Whole numbers / 整数	`100`
Float / 浮点数	`float`	Decimal numbers / 小数	`10.7`
Boolean / 布尔值	`bool`	`True` or `False` (Capitalized!)	`True`
String / 字符串	`str`	Text data / 引号内的文本	`"Hello"`

1.2.2 The type() Function / type() 函数

【Concept / 核心概念】Determining Data Type / 确定数据类型

The `type()` function is used to determine the data type of a given value or variable.
`type()` 函数用于判断任何值或变量的数据类型。

【Example / 实战案例】Using type() / type() 实操

```python
num = 100               # An integer variable
length = 10.7           # A floating point variable

print(type(num))        # <class 'int'>
print(type(length))     # <class 'float'>
```

1.2.3 Boolean Data Type / 布尔类型

【Concept / 核心概念】True or False / 真与假

The boolean data type (`bool`) can have only one of two values: `True` or `False` (both must be **capitalized**!).
布尔型只有两个值：`True`（真）和 `False`（假），首字母**必须大写**！
Booleans are fundamental for conditional expressions —we will explore them more in later chapters.

【Example / 实战案例】Boolean Examples / 布尔值示例

```
is_finished = True
print(type(is_finished))    # <class 'bool'>

is_finished = False
print(type(is_finished))    # <class 'bool'>
```

1.3 Strings: Complete Guide / 字符串完全指南

1.3.1 Properties & Creation / 属性与创建

【Concept / 核心概念】Three Key Properties / 三大核心属性

Strings in Python are identified as a contiguous set of characters represented in quotation marks. They have three important properties:

1. **Characters / 字符**: Strings contain individual letters or symbols called characters.
2. **Length / 长度**: Strings have a length, defined as the number of characters the string contains. Use `len()` to get it.
3. **Sequence / 序列**: Characters in a string appear in a sequence —each character has a **numbered position** (index) in the string.

字符串是引号内的连续字符集。三个核心属性：(1) 由字符组成 (2) 有长度 (3) 字符按序列排列。

1.3.2 Quotes and Escape Characters / 引号与转义字符

【Warning / 避坑指南】Quote Conflicts / 引号冲突

If you use double quotes inside a string delimited by double quotes, you'll get an error:

```
# text1 = "She said, "What time is it?""  # ERROR!
```

Solutions / 解决方案:

1. Use single quotes outside, double quotes inside: `text1 = 'She said, "What time is it?"'`
2. Use the escape character \: `text1 = "She said, \"What time is it?\""`

【Example / 实战案例】Escape Character / 转义字符示例

```
text1 = 'She said, "What time is it?"'
text2 = "She said, \"What time is it?\""
print(text1)    # She said, "What time is it?"
print(text2)    # She said, "What time is it?"
```

1.3.3 Determine the Length of a String / 获取字符串长度

【Example / 实战案例】Using len() / len() 函数

`len()` is a Python built-in function that returns the number of characters in a string (including spaces).

```
text2 = "This is a test string"
print(len(text2))   # Output: 22
```

1.3.4 Multiline Strings / 多行字符串

【Concept / 核心概念】Two Ways to Create Multiline Strings / 两种多行字符串方式

To deal with long strings, you can break them up across multiple lines:

1. **Backslash method / 反斜杠法**: Put a backslash (\) at the end of each line except the last.
2. **Triple quotes method / 三引号法**: Use triple quotes (""" or ''') as delimiters.

【Example / 实战案例】Multiline String Examples / 多行字符串示例

```
# Method 1: Backslash / 反斜杠法
long_text = "This planet has--or rather had--a problem, which was \
this: most of the people living on it were unhappy for pretty much of the time."
print(long_text)

# Method 2: Triple quotes / 三引号法
long_text = """This planet has--or rather had--a problem, which was
this: most of the people living on it were unhappy for pretty much of the time."""
```

```
print(long_text)
```

1.3.5 Concatenation, Indexing, and Slicing / 拼接、索引与切片

Concatenation / 字符串拼接

【Concept / 核心概念】Joining Strings with + / 用 + 拼接字符串

You can combine, or concatenate, two strings using the + operator.

【Example / 实战案例】Concatenation Example / 拼接示例

```
string1 = "Hello"
string2 = "World"
string3 = string1 + string2
print(string3)      # Output: HelloWorld (no space!)
```

Indexing / 索引

【Concept / 核心概念】The Rule of 0 / 从 0 开始计数

In Python, counting always starts at **zero (0)**. Each character in a string has a numbered position called an **index**.
Python 计数永远从 0 开始。字符串中的每个字符都有一个称为索引的编号位置。

【Example / 实战案例】Positive and Negative Indexing / 正索引与负索引

```
text = "Python BootCamp"

# Positive index / 正索引 (从左到右, 从 0 开始)
print(text[0])      # 'P' (第一个字符)
print(text[7])      # 'B'

# Negative index / 负索引 (从右到左, 从 -1 开始)
print(text[-1])     # 'p' (最后一个字符)
print(text[-5])     # 't'
```

Slicing / 切片

【Concept / 核心概念】Extracting Substrings / 提取子字符串

You can extract a portion of a string (called a **substring**) by inserting a colon between two index numbers inside square brackets: `[start:stop]`.
Critical Rule / 关键规则: Inclusive on the left, **EXCLUSIVE** on the right.
切片格式为 `[start:stop]`，左闭右开——包含 start，不包含 stop。

【Example / 实战案例】Slicing Examples / 切片实战

```python
text = "Python BootCamp"

# Basic slicing [start:stop]
print(text[0:6])    # 'Python' (indices 0,1,2,3,4,5 — NOT 6)
print(text[7:11])   # 'Boot'

# Omit start: assumes 0
print(text[:6])     # 'Python' (same as text[0:6])

# Omit stop: goes to the end
print(text[7:])     # 'BootCamp'

# Both omitted: the whole string
print(text[:])      # 'Python BootCamp'
```

1.3.6 Immutability / 不可变性

【Warning / 避坑指南】Strings Are Immutable / 字符串不可修改

You cannot change characters once a string is created. Strings are **immutable**.
字符串一旦创建就不可修改。试图执行 `text[6] = '_'` 会抛出 `TypeError: 'str' object does not support item assignment`.
Solution / 解决方法: Most string methods return a **modified copy** —you must reassign to keep the result.

【Example / 实战案例】Immutability Demonstration / 不可变性演示

```python
name = "Emlyon"
name.upper()            # Returns "EMLYON" but does NOT change 'name'!
print(name)             # Still "Emlyon"

name = name.upper()     # Correct: reassign the result
print(name)             # Now "EMLYON"
```

1.3.7 String Methods / 字符串方法大全

【Concept / 核心概念】What are String Methods? / 什么是字符串方法？

Strings come bundled with special functions called **string methods** that you can use to manipulate strings. The dot (`.`) tells Python that what follows is the name of a method.
字符串附带特殊的函数称为字符串方法，点号 (`.`) 表示接下来是方法名。

【Example / 实战案例】Common String Methods / 常用字符串方法

```python
# 1. Case Conversion / 大小写转换
print("Python BootCamp".lower())        # 'python bootcamp'
print("Python BootCamp".upper())        # 'PYTHON BOOTCAMP'

# 2. Removing Whitespace / 去除首尾空白
print(" Python BootCamp".strip())       # 'Python BootCamp'

# 3. Counting Substrings / 统计子串出现次数
text = "Python BootCamp is the best Python program ever!"
print(text.count("Python"))             # Output: 2

# 4. Finding Substring Position / 查找子串位置
print(text.find("Python"))              # Output: 0 (first occurrence)
print(text.find("python"))              # Output: -1 (not found!)

# 5. Check Start and End / 检查首尾
print("Python BootCamp".startswith('P'))  # True
print("Python BootCamp".endswith('p'))    # False

# 6. Replace / 替换
print("Hello World".replace("World", "Python"))  # 'Hello Python'
```

【Warning / 避坑指南】find() Returns -1 When Not Found / find() 找不到返回 -1

The `find()` method returns the index of the first occurrence, or **-1** if the substring is not found. This is different from indexing with `[]`, which would raise an error.
`find()` 找不到时返回 `-1`（不是报错），与直接用 `[]` 索引不同。

1.4 The None Data Type / 特殊类型 None

【Concept / 核心概念】NoneType / 空值类型

The `None` keyword is used to define a null value, or no value at all.
`None` 用于表示” 空值” 或” 无值”。
Critical Distinctions / 关键区分:

- `None` is NOT the same as `0`
- `None` is NOT the same as `False`

- `None` is NOT the same as an empty string `""`
- `None` is a data type of its own (`NoneType`)
- Only `None` can be `None`

【Example / 实战案例】None Example / None 示例

```
my_var = None
print(my_var)        # None
print(type(my_var)) # <class 'NoneType'>
```

1.5 Operators / 运算符

1.5.1 Arithmetic Operators / 算术运算符

Operator	Operation / 操作	Example / 示例
`+`	Addition / 加法	`2 + 2 = 4`
`-`	Subtraction / 减法	`5 - 2 = 3`
`*`	Multiplication / 乘法	`3 * 2 = 6`
`/`	Division / 标准除法	`5 / 2 = 2.5`
`//`	Floor Division / 整除	`5 // 2 = 2`
`**`	Exponent / 幂运算	`3 ** 2 = 9`

【Warning / 避坑指南】/ vs // 的区别

In Python 3, `/` denotes **standard division** (always returns float), while `//` denotes **floor division** (integer division, discards the remainder).
Python 3 中，`/` 是标准除法（返回 float），`//` 是整除（丢弃余数）。

【Example / 实战案例】Arithmetic Operators Demo / 算术运算符演示

```
print(2 + 2)         # 4
print(3 * 2)         # 6
print(3 ** 2)        # 9
print(5 / 2)         # 2.5 (standard division)
print(5 // 2)        # 2 (floor division)
```

1.5.2 Arithmetic on Strings / 字符串上的算术运算

【Concept / 核心概念】Operators work differently on strings / 运算符对字符串有特殊行为

Some arithmetic operators also work on `str` type:

- `+` on strings: concatenation / 拼接
- `*` between string and int: repetition / 重复

【Example / 实战案例】String Arithmetic / 字符串运算

```
print('2' + '4')        # '24' (concatenation, not addition!)
print(2 * 'S')          # 'SS' (repetition)
# print('2' * '4')      # Error! Cannot multiply string by string
```

1.5.3 Comparison Operators / 比较运算符

【Concept / 核心概念】Comparison Produces Boolean / 比较结果总是布尔值

The result of a comparison is always a boolean value (`True` or `False`).
比较运算的结果永远是布尔值（`True` 或 `False`）。

Operator	Description / 描述	Example / 示例
`==`	Equal / 等于	`1 == 1` → `True`
`!=`	Not equal / 不等于	`2.3 != 2.313` → `True`
`<`	Less than / 小于	`2.5 < 9` → `True`
`>`	Greater than / 大于	`2.4 > 2.399` → `True`
`<=`	Less than or equal / 小于等于	`3 <= 3` → `True`
`>=`	Greater than or equal / 大于等于	`2.4 >= 2.399` → `True`

【Example / 实战案例】Comparison Examples / 比较运算示例

```
print(2.5 < 3)          # True
print(2 == 2)           # True
print(2 != 3)           # True
print(5 >= 5)           # True
print(10 < 3)           # False
```

1.5.4 Logical Operators / 逻辑运算符

【Concept / 核心概念】Combining Boolean Values / 组合布尔值

Logical operators take boolean values as input and produce either `True` or `False` as output.
逻辑运算符接收布尔值并返回布尔值。

Operator	Description / 描述	Rule / 规则
`x or y`	If either is True → True; otherwise False. / 任一为真则真	Returns `True` if at least one is `True`
`x and y`	If both are True → True; otherwise False. / 两者为真才真	Returns `True` only if **both** are `True`
`not x`	Reverses the boolean value. / 取反	Returns `False` if `x` is `True`; `True` if `x` is `False`

【Example / 实战案例】Logical Operators Demo / 逻辑运算符演示

```python
# or: True if either is True
print((2 == 3) or (4 < 5))      # True (4 < 5 is True)

# and: True only if BOTH are True
print((2 == 3) and (4 < 5))     # False (2 == 3 is False)

# not: Reverses the boolean
print(not (2 == 3))             # True (reverses False to True)
print(not True)                 # False
```

1.6 Type Casting / 类型转换

【Concept / 核心概念】Converting Between Types / 类型之间的转换

If you want to change one data type to another, use the following functions:
使用以下函数进行类型转换：`int()`, `str()`, `float()`, `bool()`.
These four data types (`int`, `float`, `str`, `bool`) are called **Primitive Types**.

【Example / 实战案例】Type Casting Examples / 类型转换示例

```python
print(int(9.9))         # 9 (truncates, does NOT round!)
print(str(5))           # '5'
print(bool(1))          # True
# print(int("Yes!"))    # ValueError: invalid literal for int()
```

【Warning / 避坑指南】String + Int = Error! / 字符串 + 整数 = 报错!

You MUST cast `str` to `int` before doing arithmetic operations:

```
# Wrong / 错误
a = 34
b = "10"
# print(a + b)        # TypeError! Cannot add int and str

# Correct / 正确
a = 34
b = "10"
print(a + int(b))       # 44
```

【Concept / 核心概念】Truthiness / 真值判定规则

Almost any value is evaluated to `True` if it has some sort of content:

- Any string is `True`, **except** empty strings (`""`).
- Any number is `True`, **except** `0`.
- `bool(0)` → `False`; `bool("")` → `False`

Falsy values / 假值: `0`, `0.0`, `""`, `None`, `False`, empty containers.

1.7 String Formatting / 字符串格式化输出（四种方法）

【Concept / 核心概念】The Goal / 目标

Suppose you have `name = "Doctor Strange"`, `heads = 2`, `arms = 3`.
You want to display: **"Doctor Strange has 2 heads and 3 arms."**
There are **four** ways to achieve this in Python.

1.7.1 Method 1: Commas in print() / 用逗号分隔

【Example / 实战案例】Comma Method / 逗号法

```
name = "Doctor Strange"
heads = 2
arms = 3
print(name, "has", str(heads), "heads and", str(arms), "arms.")
# Output: Doctor Strange has 2 heads and 3 arms.
```

Note: `print()` automatically inserts spaces between comma-separated items. You must convert numbers to strings with `str()`.

1.7.2 Method 2: Concatenation with + / 字符串拼接法

【Example / 实战案例】Concatenation Method / 拼接法

```
print(name + " has " + str(heads) + " heads and " + str(arms) + " arms.")
# Output: Doctor Strange has 2 heads and 3 arms.
```

Note: Tedious! Must manually add spaces and convert numbers to strings.

1.7.3 Method 3: f-strings (Best Practice, Python 3.6+) / f-字符串（推荐）

【Concept / 核心概念】f-strings / f-字符串格式化

Two important things to notice:

1. The string literal starts with the letter `f` before the opening quotation mark.
2. Variable names surrounded by curly braces `{` and `}` are replaced with their corresponding values **without** needing `str()`.

f-strings are only available in **Python version 3.6 and above**.

【Example / 实战案例】f-string Example / f-字符串示例

```
print(f"{name} has {heads} heads and {arms} arms.")
# Output: Doctor Strange has 2 heads and 3 arms.
```

1.7.4 Method 4: .format() Method / .format() 方法

【Example / 实战案例】.format() Method / .format() 方法示例

```
print("{} has {} heads and {} arms.".format(name, heads, arms))
# Output: Doctor Strange has 2 heads and 3 arms.
```

Curly braces `{}` act as placeholders, filled by `.format()` arguments in order.

1.7.5 Method 5 (Legacy): % Operator / % 运算符（旧式）

【Example / 实战案例】% Operator / % 运算符示例

```
print("%s has %s heads and %s arms." % (name, heads, arms))
# Output: Doctor Strange has 2 heads and 3 arms.
```

`%s` is a placeholder for string; values are provided in parentheses after `%`.

1.8 User Input / 用户输入

【Concept / 核心概念】The input() Function / input() 函数

The `input()` function gets input from the user. It:

- Displays a **prompt** (a string that tells the user what to enter).
- Waits for the user to type something and press Enter.
- **Always returns a string** —even if the user types a number!

【Example / 实战案例】input() Examples / 用户输入示例

```
# Basic usage
name = input()
print(name)

# With a prompt / 带提示信息的输入
age = input("Please enter your age: ")
print(age)              # age is a STRING, not a number!
print(type(age))        # <class 'str'>

# Convert to number if needed / 如需数字，必须转换
age = int(input("Please enter your age: "))
print(age + 1)          # Now it works as a number
```

【Warning / 避坑指南】input() Always Returns a String / input() 永远返回字符串

Even if the user types `42`, the result is the string `"42"`, NOT the integer `42`. Use `int()` or `float()` to convert.
即使用户输入 42，返回的也是字符串 `"42"`，需要用 `int()` 或 `float()` 转换后才能做数学运算。

1.9 Quick Reference / 速查表

Operation / 操作	Syntax / 语法
赋值	`x = 10`
检查类型	`type(x)`
获取长度	`len(s)`
索引（正）	`s[0]`
索引（负）	`s[-1]`
切片	`s[start:stop]`
小写	`s.lower()`
大写	`s.upper()`
去空白	`s.strip()`
计数	`s.count(sub)`
查找	`s.find(sub)`
开头判断	`s.startswith(prefix)`
结尾判断	`s.endswith(suffix)`
替换	`s.replace(old, new)`
转整数	`int(x)`
转浮点	`float(x)`
转字符串	`str(x)`
转布尔	`bool(x)`
f-字符串	`f"{var}"`
用户输入	`input(prompt)`
整除	`//`
幂运算	`**`
逻辑与	`and`
逻辑或	`or`
逻辑非	`not`
等于	`==`
不等于	`!=`

2 02 - Control Structures / 控制结构

2.1 Boolean Recap / 布尔值回顾

【Concept / 核心概念】Review from Lecture 01

Booleans are one of the data types. They can only be one of two values: `True` or `False`.
布尔型只有两个值：`True`（真）和 `False`（假）。
Booleans can result from:

- **Comparison operators**: `==`, `!=`, `<`, `>`, `<=`, `>=` (e.g., `41 > 42` → `False`).
- **Logical operators**: `and`, `or`, `not` —these act on Booleans and produce Booleans.

布尔值可以通过比较运算符和逻辑运算符产生。这是条件判断的基础。

2.2 Conditionals: If-Statements / 条件判断

2.2.1 Basic Logic / 基础逻辑

【Concept / 核心概念】What is an If-Statement? / 什么是 If 语句？

If-statements allow you to execute parts of your code based on whether a condition is met.
If 语句允许程序根据条件是否满足来决定执行哪段代码。
The Rule / 核心规则: As long as the statement after the `if` keyword evaluates to `True`, the **indented** code block will run. If the statement is `False`, all the indented parts will be skipped.
只要 `if` 后面的条件为 `True`，缩进的代码块就会运行。如果为 `False`，缩进部分会被完全跳过。

2.2.2 The Full Structure: if / elif / else

【Concept / 核心概念】Three Keywords / 三个关键字

- `if`: The first condition to check. / 第一个必检条件。
- `elif` (else if): Checks additional conditions **only if** the previous `if` (or `elif`) was `False`. / 仅在前面的条件为 `False` 时才检查额外条件。
- `else`: Runs if **none** of the above conditions were `True`. It is the fallback. / 以上所有条件都不满足时的兜底执行。

【Example / 实战案例】Price Classification / 价格分级案例

课件中通过以下案例展示了完整的 `if-elif-else` 结构：

```
price = 42
if price > 100:
    print("expensive")          # Runs if price > 100
```

```
elif price > 50:                # Checked ONLY if price <= 100
    print("moderate")           # Runs if 50 < price <= 100
else:                           # Runs if all above are False
    print("cheap")              # Runs if price <= 50
# Output: cheap
```

【Warning / 避坑指南】Execution Order / 执行顺序（重要）

In an `if-elif-else` chain:

1. Python checks each condition **from top to bottom**.
2. The **first** condition that is `True` gets executed.
3. All subsequent conditions are **skipped** —even if they would also be `True`!

从上到下检查，第一个为真的条件被执行，其余全部跳过！

2.2.3 Inline Conditionals / 行内条件判断

【Example / 实战案例】Simple If-Statement / 简单 If 判断

课件展示了只需判断单个条件的场景：

```
name = "John"
if len(name) < 3:
    print("your name is short!")    # Won't run because len("John") = 4
```

如果条件为 `False`，缩进代码块什么都不执行。

【Concept / 核心概念】Ternary Operator (One-line if/else) / 三元运算符

Python also supports a one-line conditional expression for simple true/false assignments:

```
# Syntax: value_if_true if condition else value_if_false
result = "short" if len(name) < 3 else "not short"
```

三元运算符适合简单的二选一赋值，不适合复杂逻辑。

2.3 Functions / 函数基础

2.3.1 Why Functions? / 为什么需要函数？

【Concept / 核心概念】Building Blocks of Python / Python 的积木块

Functions are the building blocks of almost every Python program. They're where the real action takes place.
函数是几乎所有 Python 程序的基本构建块。
Functions break code into smaller chunks, and are great for:

- Tasks that a program uses **repeatedly** —instead of writing the same code each time, just call the function.
- Breaking complex problems into manageable pieces.

- Making code more readable and maintainable.

【Concept / 核心概念】Built-in vs. User-Defined / 内置函数 vs 自定义函数

- **Built-in functions**: Come built into Python itself (e.g., `print()`, `len()`, `type()`).
- **User-defined functions**: Functions you create yourself to perform specific tasks.

你已经用过 `print()` 和 `len()` 等内置函数，现在你将学会定义自己的函数。

2.3.2 How Function Calls Work / 函数调用的三步流程

【Concept / 核心概念】The Three Steps of a Function Call / 函数调用的三个步骤

1. **Call / 调用**: The function is called, and any arguments are passed to the function as input.
2. **Execute / 执行**: The function executes, and some action is performed with the arguments.
3. **Return / 返回**: The function returns, and the original function call is **replaced with the return value**.

【Example / 实战案例】Understanding len() / 理解 len() 的执行过程

```
num_letters = len("emlyon")
print(num_letters)          # Output: 6
```

What happens behind the scenes:

1. `len()` is called with the argument `"emlyon"`.
2. The length of the string is calculated —the number 6.
3. `len()` returns 6 and replaces the function call with the value.
4. Python then assigns `6` to `num_letters`. Equivalent to: `num_letters = 6`

【Warning / 避坑指南】Calling Errors / 调用错误

Functions like `len()` require exactly one argument. Calling `len()` without input will raise a `TypeError`: `TypeError: len() takes exactly one argument (0 given)`

- Some functions can be called with **no** arguments.
- Some functions can take **as many arguments as you like**.
- `len()` requires **exactly one** argument.

2.3.3 Writing Your Own Functions / 定义自己的函数

【Concept / 核心概念】Two Parts of a Function / 函数的两大部分

Every function has two parts:

1. **Function Signature / 函数签名**: Defines the name of the function and any inputs (parameters) it expects. Starts with the `def` keyword.

2. **Function Body / 函数体**: Contains the code that runs every time the function is used. Must be **indented**!

【Example / 实战案例】Creating Your First Function / 创建你的第一个函数

```
def multiply(x, y):          # Function signature: name + parameters
    product = x * y          # Function body (indented!)
    return product           # Return statement: sends result back

# Calling the function
result = multiply(2, 4)      # result = 8
print(result)                # 8
```

【Concept / 核心概念】Key Points About User-Defined Functions / 自定义函数要点

- `def` keyword creates a new function. The function is assigned to a variable with the same name as the function name.
- Function names become variables, so they must follow the **same naming rules** as variables.
- You call a user-defined function just like any other function: `function_name(arguments)`.
- `return` sends a value back. If there is no `return` statement, the function returns `None`.

2.3.4 Arguments and Return Values / 参数与返回值

【Concept / 核心概念】Key Terminology / 关键术语

- **Parameter / 形参**: A variable in the function signature that defines what input the function expects (e.g., `x` and `y` in `def multiply(x, y)`).
- **Argument / 实参**: The actual value passed to the function when calling it (e.g., `2` and `4` in `multiply(2, 4)`).
- **Return Value / 返回值**: When a function is done executing, it returns a value as output. The function call is replaced by this value.

2.4 Scope / 作用域的秘密

【Concept / 核心概念】Local vs. Global / 局部与全局

Names (variable names) are unique. Each function has its own **local scope** with its own set of names. Code outside functions is in the **global scope**.
变量名是唯一的。函数内部拥有独立的" 局部作用域"，函数外部的代码属于" 全局作用域"。

【Example / 实战案例】Scope Demonstration / 作用域演示

```
x = 2                        # Global x / 全局变量

def func():
```

```
    x = 3                    # NEW local x — does NOT change global x!
    print("Inside:", x)      # Prints 3 (local x)

func()                       # Output: Inside: 3
print("Outside:", x)         # Output: 2 (global x unchanged!)
```

关键理解：函数内部的 `x = 3` 创建了一个**全新的局部变量**，不会影响全局的 `x`。

【Concept / 核心概念】Scope Hierarchy / 作用域层级

Scopes have a hierarchy. When Python encounters a variable name:

1. First, it looks in the **local scope** (inside the current function).
2. If not found, it moves up to the **global scope** (outside all functions).
3. If still not found, it raises a `NameError`.

Python 查找变量时先从局部找，找不到才去全局找。

【Example / 实战案例】Scope Lookup Example / 作用域查找示例

```
x = 5                        # Global

def func():
    y = x + 5                # x not found locally → uses global x (5)
    print("Inside, y =", y) # Output: Inside, y = 10

func()
print("Outside, x =", x)     # Output: Outside, x = 5
# print(y)                   # NameError! y only exists in func()'s local scope
```

注意：函数内部可以**读取**全局变量，但不能直接修改它（除非用 `global` 关键字）。

2.5 Loops / 循环：程序如何做重复工作

【Concept / 核心概念】Why Loops? / 为什么需要循环？

Programmers don't want to work hard —they want to work **smart**. Loops are the tool for that.
程序员不想做重复劳动——循环就是用来解决这个问题的。
Imagine your boss wants you to print numbers from 1 to 10. Without loops, you'd write `print()` ten times. With loops, it's just a few lines.
想象老板让你打印 1 到 10。没有循环你得写 10 次 `print()`，有了循环只需要几行代码。
Loops help with tasks that are similar and repetitive, from simple (printing numbers) to complex (nested loops with conditionals).

2.5.1 For Loops / For 循环

【Concept / 核心概念】For Loop Structure / For 循环结构

```
for [every_element] in [my_sequence]:
    do something
```

- `every_element`: A variable created by the loop that takes on each value in the sequence, one at a time.
- `my_sequence`: The sequence to iterate over (list, tuple, string, range, etc.).
- The loop runs once for each item in the sequence.

The range() Function / range() 函数

【Concept / 核心概念】range(start, stop) / range() 详解

`range(1, 11)` produces numbers from `1` up to (but **NOT** including) `11` —i.e., 1, 2, 3, ⋯, 10.
`range(1, 11)` 产生 1 到 10 的序列，**不包含**终点 11。
Syntax / 语法:

- `range(stop)` —starts at 0, goes up to (but not including) stop.
- `range(start, stop)` —starts at start, goes up to (but not including) stop.
- `range(start, stop, step)` —starts at start, increments by step each time.

【Example / 实战案例】Printing Numbers with for / 用 for 循环打印数字

```
for i in range(1, 11):
    print(i)                # Prints 1, 2, 3, ..., 10 (each on new line)
```

其中 `i` 是迭代变量，依次取值为 1, 2, 3, ⋯, 10。

2.5.2 While Loops / While 循环

【Concept / 核心概念】When to Use While? / 什么时候用 While？

Used when we only know **when to stop**, but not **how many times** to run.
用于只知道”什么时候停”，但不知道要运行几次的情况。
Structure / 结构:

```
while [condition]:
    do whatever
```

Before starting each iteration, the condition is checked. If `True`, the loop runs. If `False`, the loop exits.

【Example / 实战案例】While Loop vs For Loop / While 循环与 For 循环对比

```
# For loop version — finite, known iterations
for i in range(1, 11):
    print(i)
```

```
4
5 # While loop version — same result, different approach
6 counter = 1
7 while counter < 11:
8     print(counter)
9     counter = counter + 1       # MUST update counter!
```

【Warning / 避坑指南】Infinite Loops / 无限循环风险

A `while` loop could run **forever** if the condition never becomes `False`!
如果条件永远不为假，`while` 循环会无休止运行!
Key Differences / For 与 While 的主要区别:

- **For loop**: Creates a variable to iterate over a given sequence. It is **finite** —you know exactly how many times it will run.
- **While loop**: Does NOT create an iteration variable for you. You must create and update your own counter. It **could run forever** if you're not careful.

2.5.3 Control Keywords / 循环控制关键字

【Concept / 核心概念】break, continue, pass / 三大控制关键字

这三个关键字可以改变循环的正常执行流程。

【Example / 实战案例】Control Keywords Demo / 控制关键字对比

```
1 # Example: break — 立即终止整个循环
2 for i in range(1, 11):
3     if i == 5:
4         break           # Exit loop completely when i = 5
5     print(i)            # Prints: 1, 2, 3, 4 (stops at 5)
```

```
1 # Example: continue — 跳过本次循环剩余部分
2 for i in range(1, 11):
3     if i == 5:
4         continue        # Skip the rest of THIS iteration
5     print(i)            # Prints: 1, 2, 3, 4, 6, 7, 8, 9, 10 (no 5!)
```

```
1 # Example: pass — 占位符，什么都不做
2 for i in range(1, 11):
3     if i == 5:
4         pass            # Do nothing — just a placeholder
5     print(i)            # Prints: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
```

Keyword	Effect / 作用	Use Case / 使用场景
`break`	Aborts the loop entirely / 直接终止整个循环	搜索到目标后退出
`continue`	Skips the rest of the current iteration / 跳过本次循环剩余部分	跳过不需要处理的特殊情况
`pass`	A placeholder that does nothing / 占位符	预留代码位置，防止空代码块报错

2.6 Summary / 速查表

Structure / 结构	Syntax / 语法
If 判断	`if condition:`
Elif 判断	`elif condition:`
Else 兜底	`else:`
定义函数	`def name(params):`
返回结果	`return value`
For 循环	`for var in sequence:`
While 循环	`while condition:`
终止循环	`break`
跳过本次	`continue`
占位符	`pass`
Range 序列	`range(start, stop, step)`

3 03 - Data Structures / 数据结构

3.1 Introduction / 什么是数据结构

【Concept / 核心概念】Data Structure / 数据结构

A data structure models a collection of data, such as a list of numbers, a row in a spreadsheet, or a record in a database.
数据结构是对数据集合的建模。
So far, you have been working with fundamental data types like `str`, `int`, and `float`. Many real-world problems are easier to solve when simple data types are combined into more complex data structures.
Python has three built-in data structures that are the focus of this chapter: **Tuples**, **Lists**, and **Dictionaries**. (Plus **Sets** as a bonus.)

3.1.1 Recap: print() Advanced / print() 进阶参数

【Concept / 核心概念】sep and end Parameters / sep 和 end 参数

`print()` can print one or multiple objects. By default:

- Objects are separated by a whitespace (`sep=" "`).
- Output ends with a line break (`end="\n"`).

These behaviors can be changed via the `sep` and `end` parameters.

【Example / 实战案例】print() with sep and end / print() 的 sep 和 end 参数

```
# Default behavior
print("hello", "there")
# Output: hello there (separated by space, ends with newline)

# Custom sep and end
print("hello", "there", sep="", end="!")
# Output: hellothere! (no separator, ends with "!" instead of newline)
```

3.2 Tuples / 元组 (Parentheses: ())

3.2.1 What is a Tuple? / 什么是元组？

【Concept / 核心概念】Sequence Type / 序列类型

A **sequence** is an ordered list of values. Each element in a sequence is assigned an integer, called an **index**, that determines the order in which the values appear. Just like strings, the index of the first value in a sequence is 0.
序列是有序的值列表。每个元素都有一个整数索引，从 0 开始。
Tuples are ordered because their elements appear in a fixed order. The first element of `(1, 2, 3)` is 1, the second is 2, the third is 3.

3.2.2 Core Properties / 核心属性

- **Ordered / 有序**: Elements appear in a fixed sequence.
- **Heterogeneous / 异构**: Can contain different types (e.g., `(1, "John", "Red")`).
- **Index starts at 0**: 索引从 0 开始。
- **Immutable / 不可变**: Once created, you cannot change, add, or remove elements.

【Example / 实战案例】Tuple Basics / 元组基础

```
tuple1 = (1, 2, 3)
print(type(tuple1))             # <class 'tuple'>

# Heterogeneous / 异构: 可以包含不同类型
tuple2 = (1, "John", "Red")
print(tuple2)                   # (1, 'John', 'Red')

# Empty tuple / 空元组
tuple3 = ()
print(len(tuple3))              # 0
```

3.2.3 Tuple Operations / 元组操作

【Example / 实战案例】Length, Indexing, Slicing / 长度、索引、切片

```
tuple2 = (1, "John", "Red")

# Length / 长度
print(len(tuple2))              # 3

# Indexing / 索引（和字符串一样！）
print(tuple2[1])                # 'John'

# Slicing / 切片
print(tuple2[1:3])              # ('John', 'Red')
```

3.2.4 Checking Existence / 成员检查

【Example / 实战案例】The in Keyword / in 关键字

```
tuple2 = (1, "John", "Red")

if "Red" in tuple2:
    print('There is red')       # This runs
else:
    print('There is no red')
```

3.2.5 Iteration / 元组遍历

【Example / 实战案例】Tuples Are Iterable / 元组可迭代

```
tuple2 = (1, "John", "Red")
for item in tuple2:
    print(item)
# Output:
# 1
# John
# Red
```

【Warning / 避坑指南】Immutability / 不可变性的后果

Like strings, tuples are **immutable**. You cannot change the value of an element once created.
元组一旦创建就无法修改。试图执行 `tuple2[1] = "David"` 会抛出 `TypeError: 'tuple' object does not support item assignment`.

3.3 Lists / 列表 (Square Brackets: [])

3.3.1 What is a List? / 什么是列表？

【Concept / 核心概念】Another Sequence Type / 另一个序列类型

The list data structure is another sequence type in Python. Just like strings and tuples, lists contain items that are indexed by integers, starting with 0.
A list literal looks almost exactly like a tuple literal, except it uses **square brackets** `[ ]` instead of parentheses.

【Example / 实战案例】Creating Lists / 创建列表

```
mylist1 = [1, 2, 3]
print(type(mylist1))            # <class 'list'>

# Like tuples, lists can contain different types
mylist2 = [1, "John", "Red"]
print(mylist2)                  # [1, 'John', 'Red']

# Create a list from string using split()
groceries = "eggs, milk, cheese"
grocery_list = groceries.split(", ")
print(grocery_list)             # ['eggs', 'milk', 'cheese']
```

3.3.2 Mutability / 可变性（关键区别）

【Concept / 核心概念】Lists Are Mutable / 列表是可变的

Unlike tuples, lists are **mutable**. You can change the value at an index even after the list has been created.
列表是**可变**的，你可以随时修改、添加或删除其中的元素。

【Example / 实战案例】Mutability Demo / 可变性演示

```
grocery_list = ['eggs', 'milk', 'cheese']
grocery_list[1] = 'butter'
print(grocery_list)             # ['eggs', 'butter', 'cheese']
```

3.3.3 List Methods / 列表常用方法

【Concept / 核心概念】Why Methods? / 为什么需要方法？

Although you can add and remove elements with slice notation, list methods provide a more **natural and readable** way to mutate a list.
虽然可以用切片增删元素，但列表方法更加直观易读。

【Example / 实战案例】insert(), pop(), append(), extend() / 四大核心方法

```python
mylist2 = []
mylist2.insert(0, 'A')          # Insert 'A' at index 0
print(mylist2)                  # ['A']

mylist2.insert(2, 'C')          # Insert at index 2 (beyond end → appended)
print(mylist2)                  # ['A', 'C']

mylist2.insert(1, 'B')          # Insert 'B' at index 1
print(mylist2)                  # ['A', 'B', 'C']

# pop(i): Remove and RETURN the value at index i
out = mylist2.pop(2)            # Removes 'C'
print(out)                      # 'C'
print(mylist2)                  # ['A', 'B']

# append(x): Add to the end
mylist2.append('D')
print(mylist2)                  # ['A', 'B', 'D']

# extend(iterable): Add multiple elements to the end
mylist2.extend(['E', 'F'])
print(mylist2)                  # ['A', 'B', 'D', 'E', 'F']
```

【Warning / 避坑指南】pop() with Out-of-range Index / pop() 越界问题

Python raises an `IndexError` if you pass to `.pop()` an argument larger than the last index.
如果传给 `pop()` 的索引超出范围，会抛出 `IndexError`。

3.3.4 Working with Lists of Numbers / 数字列表

【Concept / 核心概念】Built-in Functions for Numbers / 数字专用内置函数

Python provides convenient built-in functions for working with lists of numbers.

【Example / 实战案例】sum(), min(), max() / 求和、最小值、最大值

```python
mylist3 = [4, 17, 42, 5]

# Traditional way / 传统方式
total = 0
for number in mylist3:
    total = total + number
print(total)                    # 68

# Pythonic way / Python 风格
print(sum(mylist3))             # 68
print(min(mylist3))             # 4
print(max(mylist3))             # 42
```

3.3.5 The Reference Pitfall / 引用陷阱（重要考点）

【Warning / 避坑指南】Memory Reference / 内存地址引用

Assigning one list to another (`list2 = list1`) does **NOT** create a copy. Both refer to the **same object in memory**!
直接赋值只是起了个”别名”，修改其中一个，另一个也会跟着变。
A variable name is really just a **reference** to a specific location in computer memory. Instead of copying all the contents, `large_cats = animals` assigns the memory location referenced by `animals` to `large_cats`. Both variables now refer to the **same object**.

【Example / 实战案例】Right Way to Copy / 正确拷贝方式

```
animals = ["lion", "tiger"]

# WRONG way / 错误方式 — 只是别名
large_cats = animals
large_cats.append("Leopard")
print(large_cats)   # ['lion', 'tiger', 'Leopard']
print(animals)      # ['lion', 'tiger', 'Leopard'] — ALSO changed!

# RIGHT way / 正确方式 — 使用切片创建独立副本
animals = ["lion", "tiger"]
large_cats = animals[:]          # Creates a NEW list with same values
large_cats.append("Leopard")
print(large_cats)   # ['lion', 'tiger', 'Leopard']
print(animals)      # ['lion', 'tiger'] — unchanged!
```

3.3.6 Nesting Lists and Tuples / 嵌套结构

【Concept / 核心概念】Lists/Tuples Can Contain Lists/Tuples / 嵌套结构

Lists and tuples can contain lists and tuples as values. A **nested list** (or nested tuple) is a list or tuple that is contained as a value in another list or tuple.
列表和元组可以互相嵌套。这在数据分析中常用于表示矩阵。

【Example / 实战案例】Nested Structures / 嵌套示例

```
my_nested_list = [[1, 2], [3, 4]]
print(my_nested_list)           # [[1, 2], [3, 4]]
print(my_nested_list[1])        # [3, 4]

# Double index notation / 双重索引
print(my_nested_list[1][0])     # 3 (first element of second sublist)
```

3.3.7 Sorting Lists / 列表排序

【Concept / 核心概念】.sort() vs sorted() / 原地排序 vs 返回新序列

- `list.sort()`: Sorts the list **in-place** (modifies the original list). Returns `None`. Available for **lists only**.
- `sorted(iterable)`: Returns a **new sorted list** from any iterable. The original is unchanged. Works on **any iterable** (lists, tuples, strings, etc.).

【Example / 实战案例】Sorting Examples / 排序示例

```
# .sort() — in-place, list only
numbers = [1, 10, 5, 3]
numbers.sort()                  # Modifies 'numbers' directly
print(numbers)                  # [1, 3, 5, 10]

# sorted() — returns new list, works on any iterable
numbers = [1, 10, 5, 3]
new = sorted(numbers, reverse=True)
print(new)                      # [10, 5, 3, 1]
print(numbers)                  # [1, 10, 5, 3] — unchanged!

# Sort by key / 按自定义规则排序
colors = ["red", "yellow", "green", "blue"]
colors.sort(key=len)            # Sort by string length
print(colors)                   # ['red', 'blue', 'green', 'yellow']
```

3.4 Dictionaries / 字典 (Curly Braces: `{k: v}`)

3.4.1 Key-Value Pairs / 键值对逻辑

【Concept / 核心概念】Mapping vs. Sequence / 映射 vs 序列

Dictionaries, like lists and tuples, store a collection of objects. However, instead of storing objects in a **sequence**, dictionaries hold information in pairs of data called **key-value pairs**.
字典通过唯一的”键”（key）来存储和查找”值”（value），而不是靠位置索引。

- **Key**: A unique name that identifies the value part of the pair. / 用于标识值的唯一名称。
- **Value**: The data associated with that key. / 与键关联的数据。
- Each key is separated from its value by a colon (`:`).
- Each key-value pair is separated by a comma (`,`).
- The entire dictionary is enclosed in curly braces `{ }`.

Sequence types (Lists/Tuples) use integers as indices. **Dictionaries** use unique keys as labels —fundamentally different!

3.4.2 Creating and Accessing Dictionaries / 创建与访问

【Example / 实战案例】Dictionary Basics / 字典基础操作

```python
# Creating a dictionary / 创建字典
capitals = {
    "France": "Paris",
    "Spain": "Madrid",
    "Italy": "Rome",
}
print(capitals)
# {'France': 'Paris', 'Spain': 'Madrid', 'Italy': 'Rome'}

# Empty dictionary / 空字典（两种方式）
dict_example = {}
dict_example = dict()

# Accessing values / 访问值
print(capitals['France'])       # 'Paris'
```

3.4.3 Adding and Removing Items / 增删条目

【Concept / 核心概念】Dictionaries Are Mutable / 字典是可变的

Like lists, dictionaries are mutable data structures. You can add and remove items dynamically.
字典是可变的，可以动态增删条目。

【Example / 实战案例】Adding and Deleting / 增删操作

```python
capitals = {"France": "Paris", "Spain": "Madrid"}

# Adding a new item / 添加新条目
capitals['Germany'] = "Berlin"
print(capitals)
# {'France': 'Paris', 'Spain': 'Madrid', 'Germany': 'Berlin'}

# Deleting an item / 删除条目
del capitals["Spain"]
print(capitals)
# {'France': 'Paris', 'Germany': 'Berlin'}
```

3.4.4 Existence Check / 存在性检查

【Warning / 避坑指南】KeyError When Key Doesn't Exist / 访问不存在的键会报错

Accessing a non-existent key raises a **KeyError**. Always use `in` to check first!
访问不存在的键会抛出 `KeyError`，务必先用 `in` 检查！

【Example / 实战案例】Safe Key Access / 安全的键访问

```python
capitals = {"France": "Paris", "Spain": "Madrid"}

# Check before accessing / 先检查再访问
if "Spain" in capitals.keys():
    print(capitals["Spain"])    # 'Madrid'

# Simpler version / 更简洁的写法
if "Spain" in capitals:         # in 默认检查 keys
    print(capitals["Spain"])
```

3.4.5 Iterating Over Dictionaries / 字典遍历

【Concept / 核心概念】Different Ways to Iterate / 多种遍历方式

Dictionaries are iterable, but looping over them is different than looping over lists or tuples.

【Example / 实战案例】Dictionary Iteration / 字典遍历方法

```python
capitals = {"France": "Paris", "Spain": "Madrid", "Italy": "Rome"}

# Method 1: Loop over keys (default) / 默认遍历键
for key in capitals:
    print(key)
# Output: France, Spain, Italy (order may vary in Python < 3.7)

# Method 2: Access both key and value / 同时遍历键和值
for country in capitals:
    print(f"The capital of {country} is {capitals[country]}")

# Method 3: Using .items() — most "Pythonic" / 最 Python 的写法
for country, capital in capitals.items():
    print(f"The capital of {country} is {capital}")
# .items() returns (key, value) tuples that can be unpacked
```

3.5 Sets / 集合 (Curly Braces: {})

【Concept / 核心概念】What is a Set? / 什么是集合？

A set is a collection which is:

- **Unordered / 无序**: Elements have no fixed position.
- **Unchangeable / 不可变内容**: You cannot change items, but you can add or remove items.
- **Unindexed / 无索引**: You cannot access items by index (no `set[0]`).
- **No Duplicates / 不重复**: Sets cannot have two items with the same value. Duplicates are automatically removed!

集合是无序、无索引、不可重复的数据结构。写入相同值会被自动去重。

【Example / 实战案例】Set Creation / 创建集合

```
myset = {"apple", "banana", "cherry"}
print(myset)                    # {'apple', 'banana', 'cherry'}
print(type(myset))              # <class 'set'>

# Duplicates are automatically removed! / 重复值自动去重
myset = {"apple", "banana", "cherry", "apple"}
print(myset)                    # {'apple', 'banana', 'cherry'} (no duplicate)
```

【Warning / 避坑指南】Empty Set Requires set() / 空集合必须用 set()

{} creates an empty **dictionary**, NOT an empty set!
{} 创建的是空**字典**，不是空集合！

```
empty_dict = {}                 # This is a dict!
empty_set = set()               # This is a set!
print(type(empty_set))          # <class 'set'>
```

【Example / 实战案例】Set Operations / 集合运算

集合最大的优势在于数学集合运算——去重、交集、并集、差集等操作一行搞定：

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# Add/Remove elements / 添加与删除
A.add(10)                       # {1, 2, 3, 4, 10}
A.remove(10)                    # {1, 2, 3, 4} — KeyError if not found
A.discard(99)                   # Safe remove — no error if missing

# Union / 并集：A 或 B 中的所有元素
print(A | B)                    # {1, 2, 3, 4, 5, 6}
print(A.union(B))               # Equivalent

# Intersection / 交集：同时在 A 和 B 中的元素
print(A & B)                    # {3, 4}
print(A.intersection(B))        # Equivalent

# Difference / 差集：在 A 中但不在 B 中的元素
print(A - B)                    # {1, 2}

# Symmetric Difference / 对称差：仅在 A 或仅在 B（不同时存在）
print(A ^ B)                    # {1, 2, 5, 6}

# Subset & Superset / 子集与超集
print({1, 2} <= A)              # True — {1, 2} is subset of A
print(A >= {1, 2})              # True — A is superset of {1, 2}

```

```python
# Practical use: deduplicate a list / 实战：列表去重
duplicates = [1, 2, 2, 3, 3, 3, 4]
unique = list(set(duplicates))      # [1, 2, 3, 4] — 一行去重!
```

3.6 How to Pick a Data Structure? / 如何选择数据结构？

Use a List / 用列表	Use a Tuple / 用元组	Use a Dictionary / 用字典
Data has a natural order / 数据有自然顺序	Data has a natural order / 数据有自然顺序	Data is unordered, or order does not matter / 数据无序
You need to update/alter data / 需要更新修改数据	You will NOT update/alter data / 不需要修改数据	You need to update/alter data / 需要更新修改数据
Primary purpose is iteration / 主要用于遍历	Primary purpose is iteration / 主要用于遍历	Primary purpose is looking up values / 主要用于按键查找

3.7 List Comprehensions / 列表推导式

【Concept / 核心概念】Short-hand for Loops / for 循环的优雅简写

A list comprehension is a succinct way to create a list from an existing iterable. It is a **short-hand for a for loop**.
列表推导式是 `for` 循环的优雅简写形式，一行代码就能完成”遍历 + 转换 + 过滤”。

【Example / 实战案例】Comprehension vs Traditional Loop / 推导式 vs 传统循环

```python
numbers = (1, 2, 3, 4, 5)

# Traditional for loop / 传统 for 循环
squares = []
for num in numbers:
    squares.append(num**2)
print(squares)                  # [1, 4, 9, 16, 25]

# List comprehension / 列表推导式（一行搞定！）
squares = [num**2 for num in numbers]
print(squares)                  # [1, 4, 9, 16, 25]
```

【Example / 实战案例】Comprehension with Type Conversion / 推导式 + 类型转换

列表推导式常用于将列表中的值转换为不同类型：

```
# Convert strings to floats / 字符串列表 → 浮点数列表
str_numbers = ["1.5", "2.3", "5.25"]
float_numbers = [float(value) for value in str_numbers]
print(float_numbers)              # [1.5, 2.3, 5.25]
```

【Concept / 核心概念】Comprehension with Condition / 带条件的推导式

You can add an `if` condition at the end to **filter** elements, and the same syntax extends to dictionaries and sets.
推导式支持条件过滤，且不限于列表——字典推导式和集合推导式同样简洁。

【Example / 实战案例】Filtered & Dict/Set Comprehension / 条件过滤 + 字典/集合推导式

```
numbers = [1, 2, 3, 4, 5, 6]

# List comprehension with condition / 带过滤的列表推导式
even_squares = [num**2 for num in numbers if num % 2 == 0]
print(even_squares)                       # [4, 16, 36]

# if-else inside comprehension (ternary) / 条件表达式在左侧
labels = ["even" if n % 2 == 0 else "odd" for n in numbers]
print(labels)                             # ['odd', 'even', 'odd', 'even', ...]

# Dictionary comprehension / 字典推导式
squares_dict = {num: num**2 for num in numbers}
print(squares_dict)                       # {1: 1, 2: 4, 3: 9, 4: 16, ...}

# Dict comprehension with filter / 带过滤的字典推导式
even_squares_dict = {n: n**2 for n in numbers if n % 2 == 0}
print(even_squares_dict)                  # {2: 4, 4: 16, 6: 36}

# Set comprehension / 集合推导式
unique_lengths = {len(str(n)) for n in numbers}
print(unique_lengths)                     # {1} — all single-digit lengths are 1

# Nested comprehension / 嵌套推导式（展开二维列表）
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [x for row in matrix for x in row]
print(flattened)                          # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

3.8 Quick Reference / 速查表

Operation / 操作	Syntax / 语法
创建元组	`t = (1, 2, 3)`
创建列表	`lst = [1, 2, 3]`
创建字典	`d = {"a": 1}`
创建集合	`s = {1, 2, 3}` / `s = set()`
列表追加	`lst.append(x)`
列表插入	`lst.insert(i, x)`
列表弹出	`lst.pop(i)`
列表扩展	`lst.extend(iterable)`
列表排序 (原地)	`lst.sort()`
列表排序 (返回新)	`sorted(iterable)`
字典取值	`d["key"]` / `d.get("key")`
字典添加	`d["new_key"] = value`
字典删除	`del d["key"]`
检查键存在	`"key" in d`
字典遍历	`for k, v in d.items():`
列表推导式	`[expr for var in iterable]`
带条件推导式	`[expr for var in iterable if cond]`
print 分隔符	`print(x, y, sep=",")`
print 结尾符	`print(x, end="!")`

4 04 - Modules and Packages / 模块与包

4.1 Modules and Importing / 模块与导入

【Concept / 核心概念】What is a Module? / 什么是模块？

A module is a file containing Python code that can be re-used in other Python code files.
模块是一个包含 Python 代码的文件，可以在其他代码中重复使用。
There are libraries already included with Python and others we have to install before being able to import and use them.

4.1.1 Four Import Variations / 四种导入方式

【Concept / 核心概念】Import Syntax / 导入语法

Check the following four variations for importing modules from packages:

1. `import [package]` —imports the entire package.
2. `import [package] as [other_name]` —imports with an alias (shorter name).
3. `from [package] import [module]` —imports only a specific module/function.
4. `from [package] import [module] as [other_name]` —imports a specific item with alias.

【Example / 实战案例】Import Examples / 导入示例

```
# 1. Basic import
import numpy
arr = numpy.array([1, 2, 3])

# 2. Import with alias (most common for NumPy/Pandas)
import numpy as np
arr = np.array([1, 2, 3])

# 3. Import specific function
from math import sqrt
print(sqrt(16))             # 4.0

# 4. Import specific function with alias
from math import sqrt as square_root
print(square_root(16))      # 4.0
```

4.2 Standard Library / 标准库核心模块

【Concept / 核心概念】What is the Standard Library? / 什么是标准库？

The **Standard Library** doesn't need to be installed separately —it comes with your Python installation. These are often called **inbuilt packages**.
标准库随 Python 安装自带，无需额外安装。也叫" 内置包"。
External libraries like NumPy and Pandas must be installed before using them.

4.2.1 Copy Module / 复制模块

【Concept / 核心概念】Assignment vs Copy / 赋值与复制的区别

Assignment (=) does not copy objects; it creates a **binding** (reference) between a name and an object. For mutable collections, a copy is sometimes needed so one can change one copy without changing the other.
赋值只是创建引用（别名），不是真正的复制。对于可变集合，有时需要真正的副本来独立修改。

- **Shallow Copy / 浅拷贝** (`copy.copy`): Creates a new container, but elements inside still reference the original objects.
- **Deep Copy / 深拷贝** (`copy.deepcopy`): Recursively copies everything —completely independent from the original.

【Example / 实战案例】Shallow Copy Example / 浅拷贝案例

```
import copy
animals = ["lion", "tiger"]

# Without copy — just a reference
large_cats = animals
large_cats.append("Leopard")
print(animals)      # ['lion', 'tiger', 'Leopard'] — CHANGED!

# With copy.copy — independent copy
animals = ["lion", "tiger"]
large_cats = copy.copy(animals)
large_cats.append("Leopard")
print(large_cats)   # ['lion', 'tiger', 'Leopard']
print(animals)      # ['lion', 'tiger'] — unchanged!
```

【Example / 实战案例】Deep Copy Example / 深拷贝案例

```
import copy

# Nested list — shallow copy still shares inner objects
original = [[1, 2, 3], [4, 5, 6]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)

# Modify inner element of shallow copy
```

```
shallow[0][0] = 999
print(original)          # [[999, 2, 3], [4, 5, 6]] — CHANGED!
print(shallow)           # [[999, 2, 3], [4, 5, 6]]
print(deep)              # [[1, 2, 3], [4, 5, 6]] — unchanged!

# Modify inner element of deep copy
deep[1][0] = 777
print(original)          # [[999, 2, 3], [4, 5, 6]] — unchanged!
print(deep)              # [[1, 2, 3], [777, 5, 6]] — independent!

# Key insight: shallow copy copies the container only;
# deep copy recursively copies everything inside.
```

4.2.2 Math Module / 数学模块

【Concept / 核心概念】What's Inside math? / math 模块的内容

The `math` package includes mathematical constants and functions:

- **Constants / 常量**: `math.pi`, `math.e`.
- **Functions / 函数**: `log`, `cos`, `sin`, `ceil` (向上取整), `floor` (向下取整).

【Example / 实战案例】math Module Examples / math 模块示例

```
import math

print(math.log(5, 2))       # Logarithm base 2 of 5
print(math.pi)              # 3.141592653589793
print(math.floor(2.6))      # 2 (round down)
print(math.ceil(2.6))       # 3 (round up)
```

4.2.3 Time Module / 时间模块

【Concept / 核心概念】What's Inside time? / time 模块的内容

The `time` package deals with time and timing-related things.
`time.time()` returns the seconds that have passed since **January 1st 1970 00:00:00** (the Unix epoch).

【Example / 实战案例】Time Module Examples / time 模块示例

```
import time

# Get current Unix timestamp / 获取当前时间戳
print(time.time())          # e.g., 1717718400.123456

# Pause execution for 2 seconds / 暂停 2 秒
print("Wait...")
time.sleep(2)
```

```
print("Hi")                  # Printed after 2 seconds

# Timing how long a function takes / 测量函数执行时间
def my_function():
    for n in range(1000000):
        n += 2 * n

start_time = time.time()
my_function()
elapsed = time.time() - start_time
print(f"{elapsed:.4f} seconds")
```

【Example / 实战案例】Time Object with localtime() / 本地时间对象

```
now = time.localtime()
print(now)                           # time.struct_time object
print(now.tm_year)                   # Current year, e.g., 2026
print(now.tm_mon)                    # Current month
print(now.tm_mday)                   # Current day
```

4.3 External Libraries & Pip / 外部库与包管理

【Concept / 核心概念】Why Pip? / 为什么需要 Pip？

External packages need to be installed before using them. They are not included with Python by default. `pip` is the package manager for Python —it automates installing, upgrading, and removing third-party packages.
`pip` 是 Python 的包管理器，用于安装、升级和卸载第三方包。
`pip` is a command line tool —you must run it from a terminal.

【Example / 实战案例】Common pip Commands / 常用 pip 命令

```
# Check pip version / 检查 pip 版本
pip --version

# Upgrade pip itself / 升级 pip 自身
pip install --upgrade pip

# List all installed packages / 列出已安装的所有包
pip list

# Install a package / 安装包
pip install numpy

# Install a specific version / 安装特定版本
pip install numpy==1.16.5

# Uninstall a package / 卸载包
pip uninstall numpy
```

4.4 Numerical Computing with NumPy / NumPy 数值计算

4.4.1 Why NumPy? / 为什么 NumPy 这么快？

【Concept / 核心概念】The Type-Checking Bottleneck / 类型检查瓶颈

Python lists can store any object, so the interpreter must check the data type for **every** operation. NumPy uses **ndarrays** with a **fixed data type**, skipping these checks and using **vectorized operations**.
Python 列表因为支持异构数据，每次计算都要检查类型。NumPy 通过固定数据类型（`dtype`）并使用向量化计算，速度快几十倍。

【Example / 实战案例】Speed Comparison / 速度对比

```python
import random
import numpy as np

# Create 1,000,000 random integers with Python list
measurements = [random.randint(150, 200) for _ in range(1_000_000)]

# NumPy version
measurements_array = np.array(measurements)

# NumPy's vectorized mean is MUCH faster
np.mean(measurements_array)     # ~50x faster than sum/len on list
```

4.4.2 Creating NumPy Arrays / 创建 NumPy 数组

【Concept / 核心概念】Multiple Ways to Create Arrays / 多种创建方式

- **From list**: `np.array([10, 2, 35])`
- **arange**: Like `range()` but returns ndarray. `np.arange(start=2, stop=14, step=2)`
- **linspace**: Creates array with N evenly spaced values. `np.linspace(start=-5, stop=5, num=9)`
- **zeros / ones**: Arrays filled with 0 or 1. `np.zeros(3)`, `np.ones((2, 3))`

【Example / 实战案例】Creating Arrays / 创建数组案例

```python
import numpy as np

# From list
print(np.array([10, 2, 35]))            # [10  2 35]

# arange: like range but NumPy
print(list(range(5)))                   # [0, 1, 2, 3, 4]
print(np.arange(start=2, stop=14, step=2))  # [ 2  4  6  8 10 12]
```

```python

# linspace: N evenly spaced values
print(np.linspace(start=-5, stop=5, num=9))
# [-5.   -3.75 -2.5 -1.25  0.    1.25  2.5  3.75  5. ]

# zeros and ones
print(np.zeros(3))                      # [0. 0. 0.]
print(np.ones(3))                       # [1. 1. 1.]

# Multidimensional with shape argument
print(np.zeros((2, 2)))                 # 2x2 matrix of zeros
# [[0. 0.]
#  [0. 0.]]
print(np.ones((2, 3)))                  # 2x3 matrix of ones
```

4.4.3 Anatomy of Arrays / 数组解剖

【Concept / 核心概念】dtype, shape, ndim / 三大核心属性

- `.dtype`: Data type of the array elements. Can be `int`, `float`, `bool`, etc.
- `.shape`: A tuple with the number of elements in each dimension. E.g., `(3, 4)` for a 3×4 array.
- `.ndim`: The total number of dimensions.

【Example / 实战案例】Array Anatomy / 数组属性详解

```python
import numpy as np

# dtype — data type
values = [0, 1, 2, 3, 4]
int_arr = np.array(values, dtype='int')
print(int_arr, int_arr.dtype)           # [0 1 2 3 4] int64

# NumPy casts values to match dtype
bool_arr = np.array(values, dtype='bool')
print(bool_arr, bool_arr.dtype)
# [False  True  True  True  True] bool

# Without explicit dtype: "smallest common denominator"
values = [0, 1, 2.5, 3, 4]
float_arr = np.array(values)
print(float_arr, float_arr.dtype)       # all become float64

# 1D Array
one_dim_arr = np.array([1, 2, 3, 4])
print(one_dim_arr.shape)                # (4,)
print(one_dim_arr.ndim)                 # 1

# 2D Array
values = [[1, 2, 3, 4, 5],
```

```
          [1, 2, 3, 4, 1],
          [1, 2, 3, 4, 2]]
two_dim_arr = np.array(values)
print(two_dim_arr.shape)                # (3, 5)
print(two_dim_arr.ndim)                 # 2

# Access 2D elements: [row, column]
print(two_dim_arr[1, 3])                # 4
two_dim_arr[1, 3] = 10                 # Lists are mutable!
print(two_dim_arr)

# 3D Array
values = [[[1, 2, 3, 4]] * 3] * 6
three_dim_arr = np.array(values)
print(three_dim_arr.shape)              # (6, 3, 4)
print(three_dim_arr.ndim)               # 3
print(three_dim_arr[1, 1, 1])           # Access 3D element
```

4.4.4 Reshape / 重塑数组形状

【Example / 实战案例】Using reshape() / reshape() 用法

```
a = np.arange(start=2, stop=14)         # [2, 3, ..., 13] — 12 elements
print(a.shape)                          # (12,)

# Reshape to 3x4
print(a.reshape(3, 4))
# [[ 2  3  4  5]
#  [ 6  7  8  9]
#  [10 11 12 13]]

# -1 auto-detects the size of that dimension
print(a.reshape(2, -1))                 # 2x6
print(a.reshape(2, -1).shape)           # (2, 6)
```

4.4.5 Comparing Arrays / 数组比较

【Example / 实战案例】Equality and np.isclose() / 相等比较与 np.isclose()

```
a = np.zeros((3, 3))
b = np.zeros((3, 3))
print(a == b)                   # All True (element-wise comparison)

# Floating point precision issue!
a[0, 0] += 0.000000000000001
print(a == b)                   # One element is now False!

# Use np.isclose() for floating point comparison
```

```
print(np.isclose(a, b))          # All True (tolerates tiny differences)
```

4.4.6 Mathematical Operations / 数学运算（向量化）

【Concept / 核心概念】Vectorization / 向量化

Numpy's mathematical functions operate on arrays in a **vectorized** manner —applied to each element without explicit for-loops. Vectorized functions are called **ufuncs** (universal functions) in NumPy.
向量化操作直接作用于整个数组的每个元素，不需要写 `for` 循环。

【Example / 实战案例】Vectorized Operations / 向量化运算

```
arr = np.arange(1, 10)           # [1 2 3 4 5 6 7 8 9]

# Element-wise operations / 逐元素运算
print(arr * 3)                   # [ 3  6  9 12 15 18 21 24 27]
print(arr * arr)                 # [ 1  4  9 16 25 36 49 64 81]
print(arr + (arr * 2))           # [ 3  6  9 12 15 18 21 24 27]
print(arr - arr)                 # [0 0 0 0 0 0 0 0 0]
print(arr / arr)                 # [1. 1. 1. ...]
print(arr ** 2)                  # [ 1  4  9 16 25 36 49 64 81]

# Matrix multiplication (inner product for 1D)
print(arr @ arr)                 # 285 (= 1*1 + 2*2 + ... + 9*9)
# Same as: np.sum(arr * arr)

# Universal functions / 通用数学函数
print(np.log(arr))
print(np.log2(arr))
print(np.exp(arr))
print(np.sin(arr))
```

【Warning / 避坑指南】* vs @ / 逐元素乘法 vs 矩阵乘法

- `arr * arr`: Element-wise multiplication / 逐元素乘法.
- `arr @ arr`: Matrix product (inner product for 1D vectors) / 矩阵相乘（一维时为内积）.

混淆这两个操作会导致完全不同的结果!

4.4.7 Aggregation Functions / 聚合函数与轴

【Concept / 核心概念】Reducing Dimensionality / 降维操作

Aggregation functions (min, max, sum, average) reduce dimensionality. They provide an **axis** argument to specify which dimension to reduce.
聚合函数降低数组维度。`axis` 参数指定沿着哪个轴聚合。

【Example / 实战案例】Aggregation with axis / 聚合与轴参数

```
two_dim_arr = np.random.randint(0, high=20, size=(3, 4))
print(two_dim_arr)              # 3x4 random array

# Aggregation over whole array / 对整个数组聚合
print(np.min(two_dim_arr))      # Minimum of ALL elements
print(np.max(two_dim_arr))      # Maximum of ALL elements
print(np.sum(two_dim_arr))      # Sum of ALL elements

# axis=0: Operation applied across rows (column-wise)
# 纵向聚合：按列计算，结果 shape 为 (4,)
print(np.min(two_dim_arr, axis=0))  # Min of each column

# axis=1: Operation applied across columns (row-wise)
# 横向聚合：按行计算，结果 shape 为 (3,)
print(np.average(two_dim_arr, axis=1))  # Avg of each row
```

【Warning / 避坑指南】axis Meaning / axis 参数含义

- `axis=0`: Operation across rows —reduces to one value per **column**. / 纵向（按列计算）.
- `axis=1`: Operation across columns —reduces to one value per **row**. / 横向（按行计算）.

Think of it as: the axis you specify is the one that **disappears**!

4.4.8 Combining Arrays / 数组合并

【Example / 实战案例】concatenate, append, stack / 数组合并三剑客

```
a = np.arange(10)                   # [0 1 2 3 4 5 6 7 8 9]
b = np.arange(10)[::-1]             # [9 8 7 6 5 4 3 2 1 0]

# concatenate: joins a sequence of arrays
print(np.concatenate((a, b)))
# [0 1 2 3 4 5 6 7 8 9 9 8 7 6 5 4 3 2 1 0]

# append: uses concatenate internally
print(np.append(a, b))

# stack: stacks along a NEW axis (creates new dimension)
print(np.stack((np.arange(10), np.arange(10))))
# [[0 1 2 3 4 5 6 7 8 9]
#  [0 1 2 3 4 5 6 7 8 9]]
```

【Warning / 避坑指南】Concatenation Creates Copies / 合并操作会创建副本

Operations like `np.append`, `np.concatenate`, and `np.stack` always require the whole array to be copied. It often makes more sense to allocate an array of the size you need **upfront** and then fill it.

4.4.9 Masking / 掩码过滤

【Concept / 核心概念】Boolean Indexing / 布尔索引

Logical arrays (arrays containing boolean values) can be used to index other arrays. These logical arrays are called **masks**. This is especially useful to filter data based on conditions.
布尔数组可以用作索引来过滤数据，称为”掩码”。

【Example / 实战案例】Masking Examples / 掩码过滤示例

```
arr = np.arange(1, 6)                # [1 2 3 4 5]

# Create a boolean mask
mask = np.array([True, False, True, False, True])
print(arr[mask])                     # [1 3 5] (only where mask is True)

# Generate mask from condition
print(arr < 5)                       # [ True  True  True  True False]
print(arr[arr < 3])                  # [1 2] (filtered by condition)
```

4.4.10 Advanced Indexing / 高级索引

【Example / 实战案例】2D Array Indexing / 二维数组索引

The syntax works as `[row, column]`, or more precisely `[row_start:row_end, col_start:col_end]`.

```
large_two_dim_arr = np.arange(81).reshape((9, 9))

# Select entire column / 选择整列
print(large_two_dim_arr[:, 2])       # All rows, column 2

# Select specific row and column range
print(large_two_dim_arr[1, 2:7])     # Row 1, columns 2 through 6

# Standard slicing with step / 带步长的切片
print(large_two_dim_arr[:, 2:7:2])   # All rows, columns 2, 4, 6
```

5 05 - Object Oriented Programming / 面向对象编程

5.1 Why OOP? / 为什么需要面向对象？

【Concept / 核心概念】From Lists to Classes / 从列表到类

OOP (Object-Oriented Programming) is a method of structuring a program by bundling related properties and behaviors into individual **objects**.
面向对象编程是一种将相关属性和行为打包成”对象”的程序设计方法。
The Problem with Lists / 用列表存储数据的困境:
Imagine tracking employees in an organization. You could represent each employee as a list:

```
Eric = ["Eric", 34, "CEO", 2016]
Alice = ["Alice", "Science Officer", 2017]
```

There are issues with this approach:

1. You need to **remember** which index means what (e.g., index 0 = name, index 1 = age).
2. What if not every employee has the same number of elements? (Age is missing for Alice!)
3. What if you want to define **behaviors** (methods) for your employees?

Classes allow you to create **User-defined Data Structures** that group data (attributes) and behaviors (methods) together. Classes define a type.

5.2 Creating Classes / 创建类

5.2.1 The Blueprint / 蓝图定义

【Concept / 核心概念】The class Keyword / class 关键字

All class definitions start with the `class` keyword, followed by the name of the class and a colon (`:`).

- **Naming / 命名**: Use **CamelCase** (e.g., `Employee`), starting with a capital letter. This is different from functions/variables which use `snake_case`.
- `pass`: A placeholder used as a body where code will eventually go. Allows you to run the class without throwing an error.

【Example / 实战案例】Minimal Class / 最简类定义

```
class Employee:
    pass                # Placeholder — does nothing, but avoids SyntaxError
```

5.2.2 The `__init__()` Method / 初始化方法

【Concept / 核心概念】The Initializer / 初始化器

The `__init__()` method is a special method that runs **every time** a new object is created. It tells Python what the **initial state** (initial values of the object's properties) should be.
`__init__()` 在每次创建新对象时自动运行，设置对象的初始值。

【Warning / 避坑指南】What is self? / self 到底是什么？

The first argument of `__init__()` is **always** `self`. It is a reference to the **specific instance** being created. Think of it as a **placeholder** while building the blueprint —it refers to the instance that will be created later.
`self` 指代"当前的这个实例"。它是构建蓝图时的占位符——虽然实例还没创建，但 `self` 代表了将来会被创建的实例对象。

5.3 Attributes / 属性

【Concept / 核心概念】Instance vs Class Attributes / 实例属性 vs 类属性

- **Instance Attributes / 实例属性**: Belong to a **specific** object. Defined inside `__init__()` with `self.xxx`. Each instance has its own copy.
  例: `self.name` —每个员工有不同的名字。
- **Class Attributes / 类属性**: Same for **ALL** instances of a class. Defined directly inside the class body (outside `__init__()`). All instances share the same value.
  例: `company_name = "Emlyon"` —所有员工属于同一公司。

【Example / 实战案例】Class Definition with Attributes / 完整类定义

```python
class Employee:
    # Class Attribute / 类属性 — shared by all instances
    company_name = "Emlyon"

    def __init__(self, name, age):
        # Instance Attributes / 实例属性 — unique to each instance
        self.name = name
        self.age = age
```

5.4 Instantiating & Methods / 实例化与方法

5.4.1 Instantiating Objects / 实例化对象

【Concept / 核心概念】Blueprint → Object / 蓝图 → 对象

While the **class** is the blueprint, an **instance** is an object built from a class that contains real data.
类是蓝图，实例是从类构建出来的包含真实数据的对象。
To instantiate an object, type the name of the class (CamelCase) followed by parentheses containing any values

that must be passed to `__init__()`.

【Example / 实战案例】Creating Objects / 创建对象

```python
emp1 = Employee("David", 23)
emp2 = Employee("Alice", 35)

print(emp1)                      # <__main__.Employee object at 0x00aeff70>
# The hex number is the memory address — different on each computer!

print(emp1 == emp2)              # False (different objects in memory)
print(type(emp1))                # <class '__main__.Employee'>

# Access attributes with dot notation / 用点号访问属性
print(emp1.name)                 # 'David'
print(emp2.age)                  # 35

# Class attributes are accessed the same way
print(emp1.company_name)         # 'Emlyon'
print(emp2.company_name)         # 'Emlyon' (same for all)
```

【Warning / 避坑指南】Objects Are Mutable by Default / 对象默认是可变的

```python
emp1.age = 36                    # Dynamically modify instance attribute
print(emp1.age)                  # 36 — changed!
```

Custom objects are **mutable** by default —their attributes can be altered dynamically. This is an important takeaway.

5.4.2 Instance Methods / 实例方法

【Concept / 核心概念】Behaviors / 行为

Classes also have special functions, called **methods**, that define behaviors and actions that an object can perform with its data.
类内部的函数定义了对象能做的动作。实例方法的第一个参数永远是 `self`。

【Example / 实战案例】Instance Methods Example / 实例方法案例

```python
class Employee:
    company_name = "Emlyon"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    # Instance method / 实例方法
    def description(self):
        return f"{self.name} is {self.age} years old"
```

```python

    # Another instance method / 另一个实例方法
    def update_age(self, new_age):
        self.age = new_age

# Usage
emp1 = Employee("David", 23)
print(emp1.description())       # 'David is 23 years old'
emp1.update_age(24)
print(emp1.description())       # 'David is 24 years old'
```

5.4.3 Dunder Methods / 魔术方法

【Concept / 核心概念】The ___str___() Method / ___str___() 方法

By default, `print(emp1)` outputs something ugly like `<__main__.Employee object at 0x00aeff70>`. You can change what gets printed by defining a special instance method called `__str__()`.
定义 `__str__()` 方法可以改变打印对象时的显示内容。

【Example / 实战案例】Customizing print() Output / 自定义 print() 输出

```python
class Employee:
    company_name = "Emlyon"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old"

emp1 = Employee("David", 23)
print(emp1)                     # 'David is 23 years old' — nice!
```

【Concept / 核心概念】What Are Dunder Methods? / 什么是魔术方法？

Methods like `__str__()` are called **dunder methods** because they begin and end with **d**ouble **under**scores. There are many dunder methods available that allow your classes to work well with other Python language features (e.g., `__init__`, `__str__`, `__repr__`, `__eq__`, `__len__`).

5.5 Inheritance / 继承

【Concept / 核心概念】Parent vs Child / 父类与子类

Inheritance is the process by which one class takes on the attributes and methods of another.
继承允许一个类获取另一个类的所有属性和方法。

- **Parent Class / 父类**: The class being inherited from.
- **Child Class / 子类**: The class that inherits. Child classes can **override** (redefine) and **extend** (add to) the parent's attributes and methods.
- To create a child class, put the parent class name in parentheses after the child class name.

【Concept / 核心概念】The Function / super() 函数

`super()` calls the **parent** class's methods, especially useful for calling `__init__()` in the child class to initialize inherited attributes.
`super()` 用于调用父类的方法，最常见的用法是在子类的 `__init__()` 中调用父类的 `__init__()`。

【Example / 实战案例】Full Inheritance Example / 继承完整案例

```python
# Parent class / 父类
class Employee:
    company_name = "Emlyon"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old"

# Child class 1: HR Employee / 子类 1
class HREmployee(Employee):
    def __init__(self, name, age, wh):
        super().__init__(name, age)      # Call parent's __init__
        self.working_hours = wh          # New attribute / 新属性

    def update_working_hours(self, new_hour):   # New method / 新方法
        self.working_hours = new_hour

    def __str__(self):                           # Override __str__ / 重写
        return f"{self.name} is {self.age} years old and works in HR and has {self.
            working_hours} working hours!"

# Child class 2: Management Employee / 子类 2
class ManagementEmployee(Employee):
    def __init__(self, name, age):
        super().__init__(name, age)
        self.emp_list = []               # New attribute / 新属性

    def add_employee(self, emp_name):    # New method / 新方法
```

```
        self.emp_list.append(emp_name)

    def give_emp_list(self):            # New method / 新方法
        for i in range(len(self.emp_list)):
            print(f"{i} : {self.emp_list[i]}")
```

【Example / 实战案例】Using the Child Classes / 使用子类

```
# HR Employee / HR 员工
hr_emp_1 = HREmployee("David", 23, 8)
print(hr_emp_1)                     # Uses overridden __str__
hr_emp_1.update_working_hours(10)   # Child's own method
print(hr_emp_1.working_hours)       # 10

# Management Employee / 管理员工
management_emp_1 = ManagementEmployee("Alice", 35)
print(management_emp_1)             # Uses parent's __str__
management_emp_1.add_employee("John")
management_emp_1.add_employee("Maria")
management_emp_1.give_emp_list()
# Output:
# 0 : John
# 1 : Maria
```

5.6 Encapsulation & Properties / 封装与属性装饰器

【Concept / 核心概念】Read-Only Attributes / 只读属性

Sometimes it's a bad idea if someone can change an attribute from outside the class, but you still want to **see** the value. This is done using a **getter-function** with the `@property` decorator. To allow **controlled** modification (e.g., with validation), use `@xxx.setter`.
使用 `@property` 装饰器可以将方法伪装成属性，实现只读访问。使用 `@xxx.setter` 可以在赋值时加入验证逻辑。

【Example / 实战案例】@property Example / @property 案例

```
class MyClass:
    def __init__(self):
        self._hidden = 42           # Prefix '_' suggests internal use

    @property
    def hidden(self):               # Getter — can access like an attribute
        return self._hidden

    def get_hidden(self):           # Ordinary getter — requires parentheses
        return self._hidden

my_object = MyClass()
```

```python

# @property: access like an attribute (no parentheses!)
print(my_object.hidden)             # 42

# Direct assignment to @property is BLOCKED
# my_object.hidden = 0              # AttributeError! Read-only!

# BUT: _hidden can still be accessed directly if you really want
my_object._hidden = 0               # Works, but you shouldn't do it
print(my_object._hidden)            # 0

# Ordinary getter requires parentheses
print(my_object.get_hidden())       # 0
```

【Example / 实战案例】@property.setter Example / @property.setter 案例（带验证的写入）

通过 `@xxx.setter` 可以在赋值时自动执行验证逻辑——阻止非法值写入：

```python
class Student:
    def __init__(self, name, score):
        self.name = name
        self._score = score             # Internal storage

    @property
    def score(self):                    # Getter — read like attribute
        return self._score

    @score.setter
    def score(self, value):             # Setter — validate before writing
        if value < 0 or value > 100:
            raise ValueError("Score must be between 0 and 100!")
        self._score = value

s = Student("Alice", 85)
print(s.score)          # 85 — reads like attribute (no parentheses)

s.score = 92            # Valid — setter silently accepts
print(s.score)          # 92

# s.score = 150         # ValueError: Score must be between 0 and 100!
# s.score = -10         # ValueError — blocked by setter validation!

# Key advantage: the validation is REUSABLE — every assignment
# to .score automatically goes through the setter's check.
```

【Warning / 避坑指南】The _ Prefix Convention / _ 前缀约定

The underscore prefix (`_hidden`) is a **convention** that tells other programmers "this attribute is internal—don't access it directly." However, it does **NOT** actually prevent access. Python trusts you to follow the convention.

5.7 Quick Reference / 速查表

Concept / 概念	Syntax / 语法
定义类	`class MyClass:`
初始化方法	`def __init__(self, param):`
实例属性	`self.attr = value`
类属性	`class_attr = value`（在 `__init__` 外）
实例化	`obj = MyClass(arg)`
字符串表示	`def __str__(self): return "..."`
继承	`class Child(Parent):`
调用父类	`super().method()`
只读属性	`@property` 装饰 getter 方法
内部属性约定	`self._internal`

6 06 - File I/O & Error Handling / 文件 IO 与错误处理

6.1 File Systems & Paths / 文件系统与路径

【Concept / 核心概念】Hierarchy and Paths / 层级与路径

File systems organize files in a hierarchy of **directories** (folders). At the top is the **root directory**.
文件系统将文件组织在目录（文件夹）的层级结构中。最顶层是根目录。
Each file in a directory has a file name that must be unique from any other file in the same directory. Directories can contain **subdirectories** (subfolders).
A **file path** is a string representing the location of a file, starting from the root directory.
文件路径是从根目录开始定位文件的字符串。

【Warning / 避坑指南】OS Differences / 操作系统路径差异

How you write file paths depends on your operating system:

- **Windows**: Uses backslashes (\), e.g., `C:\Users\David\Documents\hello.txt`.
- **macOS**: Uses forward slashes (/), e.g., `/Users/David/Documents/hello.txt`.
- **Ubuntu Linux**: Uses forward slashes (/), e.g., `/home/David/Documents/hello.txt`.

6.2 The open() Function / 打开文件

6.2.1 Opening Modes / 打开模式

【Concept / 核心概念】Creating a File Object / 创建文件对象

Before you can read or write a file, you must open it using Python's built-in `open()` function. This function creates a **file object**.
读写文件之前必须先用 `open()` 打开，这会创建一个文件对象。
`open(file='path/to/file', mode='...')`

- `'r'` **(Read)**: Read only (default if mode is omitted). File must exist. / 只读。
- `'w'` **(Write)**: Overwrites existing file or creates a new one. **WARNING**: erases existing content! / 覆写（会清空原内容）。
- `'a'` **(Append)**: Adds content to the end of the file. Creates the file if it doesn't exist. / 追加（在末尾添加内容）。

6.2.2 File Attributes / 文件属性

【Example / 实战案例】Checking File Attributes / 查看文件属性

```
my_file = open(file='sample.txt', mode='w')

print("Name of the file: ", my_file.name)     # sample.txt
print("Closed or not: ", my_file.closed)       # False
print("Opening mode: ", my_file.mode)          # w
```

6.2.3 Writing to Files / 写入文件

【Example / 实战案例】write() Method / write() 方法

The `write()` method writes any string to an open file. **IMPORTANT**: It does **NOT** add a newline character (`\n`) automatically —you must add it manually!
`write()` 不会自动添加换行符，必须手动加 `\n`!

```
my_file = open(file='sample.txt', mode='w')
my_file.write('Hello file!')
my_file.write('\n')                    # Manually add newline
my_file.write('This is me!')
my_file.close()                        # Always close to flush data!
```

6.2.4 The close() Method / 关闭文件

【Concept / 核心概念】Why close()? / 为什么必须 close()？

The `close()` method flushes any unwritten information and closes the file object, after which no more writing can be done.
`close()` 刷写缓冲区中的未保存数据并关闭文件。不关闭可能导致数据丢失!

【Example / 实战案例】The with Statement (Best Practice) / with 语句（最佳实践）

Recommended: Use `with open(...) as f:` —it automatically closes the file, even if an exception occurs. This avoids the common mistake of forgetting `close()`.
推荐：使用 `with` 语句自动关闭文件，即使发生异常也不会泄漏资源。

```
# OLD WAY — must remember to close (easy to forget!)
my_file = open('sample.txt', 'w')
my_file.write('Hello!\n')
my_file.close()                          # Don't forget this!

# BETTER WAY — with statement auto-closes!
with open('sample.txt', 'w') as f:
    f.write('Hello!\n')                  # No need to call close()
    f.write('World!\n')
# File is automatically closed here — even if an error occurred!

```

```python
# Reading with `with` + iterating
with open('sample.txt', 'r') as f:
    for line in f:                      # Memory-efficient iteration
        print(line.strip())             # Strip removes trailing \n

# `with` is the Pythonic way — it works with ALL file modes
# and guarantees cleanup. Use it in every exam answer!
```

6.3 Reading Files / 读取文件

【Concept / 核心概念】Four Ways to Read / 四种读取方式

- `.read()`: Reads the **entire** file as one single string. / 整个文件读入一个字符串。
- `.readline()`: Reads just **one line** (up to and including the newline character). / 仅读取一行。
- `.readlines()`: Reads the whole file and returns a **list of strings** —each line is one element. / 返回字符串列表，每个元素是一行。
- `for line in file`: Iterates line by line —the most **memory-efficient** way! / 逐行遍历，最省内存。

【Example / 实战案例】Reading Methods Comparison / 四种读取方式对比

```python
# Method 1: .read() — entire file as one string
my_file = open(file='sample.txt', mode='r')
file_output = my_file.read()
my_file.close()
print(file_output)

# Method 2: .readline() — just one line
my_file = open(file='sample.txt', mode='r')
file_output = my_file.readline()    # Reads first line
my_file.close()

# Method 3: .readlines() — list of lines
my_file = open(file='sample.txt', mode='r')
file_output = my_file.readlines()
my_file.close()
print(file_output)                  # ['Hello file!\n', 'This is me!\n']

# Method 4: for line in file — BEST for large files!
my_file = open(file='sample.txt', mode='r')
for line in my_file:                # Most memory-efficient
    print(line)
my_file.close()
```

6.4 Working with CSV Files / CSV 文件处理

【Concept / 核心概念】What is CSV? / 什么是 CSV？

CSV stands for **Comma Separated Values**. It is a simple but powerful way to store data in a table format.
CSV 是一种用逗号分隔值的简单表格数据格式。

6.4.1 Reading CSV / 读取 CSV

【Example / 实战案例】Using csv.reader() and DictReader / CSV 读取器

```python
import csv

# Method 1: csv.reader — returns lists
csv_file = open("sample.csv", "r")
reader = csv.reader(csv_file, skipinitialspace=True)
for row in reader:
    print(row)                      # ['1', 'Rose', '30', 'Grenoble']
csv_file.close()

# Method 2: csv.DictReader — returns dicts (RECOMMENDED!)
csv_file = open("sample.csv", "r")
reader = csv.DictReader(csv_file)
for row in reader:
    print(row)
    # {'id': '1', 'name': 'Rose', 'age': '30', 'city': 'Grenoble'}
csv_file.close()
```

6.4.2 Writing CSV / 写入 CSV

【Example / 实战案例】Using csv.writer() / CSV 写入器

```python
# Write CSV — use "w" to overwrite, "a" to append
csv_file = open("sample.csv", "w")
writer = csv.writer(csv_file)
writer.writerow(['5', 'Rose', '30', 'Grenoble'])
writer.writerow(['6', 'Nona', '23', 'Toulouse'])
csv_file.close()
```

【Warning / 避坑指南】'w' vs 'a' for CSV / CSV 写入模式选择

If you do NOT want to overwrite your existing file, use `'a'` (append) instead of `'w'` (write).
用 `'w'` 会清空原文件内容，用 `'a'` 会在末尾追加！

6.5 Error Handling & Debugging / 异常处理与调试

6.5.1 Error Types / 错误类型

【Concept / 核心概念】Errors vs Exceptions / 语法错误 vs 运行时异常

- **SyntaxError / 语法错误**: Caught by the IDE **before** execution. The script cannot run at all. Examples: forgetting a colon `:`, forgetting to close a bracket, invalid variable names.
- **Exceptions / 运行时异常**: Occur **during** execution. Examples: `IndexError` (list index out of range), `NameError` (variable not defined), `TypeError` (wrong type operation).

【Example / 实战案例】Common Error Examples / 常见错误类型

```python
# SyntaxError: invalid name (starts with digit)
# 3_apples = ["apple", "apple", "apple"]

# SyntaxError: forgot to close brackets
# three_apples = ["apple", "apple", "apple"

# SyntaxError: forgot colon and wrong indentation
# for i in range(3)
# print(three_apples[i])

# IndexError: index out of range
three_apples = ['A', 'B', 'C']
# print(three_apples[4])          # IndexError!

# TypeError: cannot concatenate str + int
# print("hello " + 6)             # TypeError!

# ValueError: invalid literal for float()
converting_data_types = "I am a string"
# print(float(converting_data_types))  # ValueError!
```

6.5.2 The Try-Except Block / 捕获异常

【Concept / 核心概念】try-except Logic / try-except 逻辑

`try-except` acts like `if-else` but for exceptions. Python's full exception-handling structure has 4 blocks:

- `try`: Code that *might* raise an exception. / 可能出错的代码。
- `except [ExceptionType]`: Runs when the specified exception occurs. **Always specify the exception type!** A bare `except:` catches everything (including keyboard interrupts) and hides bugs. / 捕获指定类型的异常。
- `else`: Runs **only if no exception** occurred. Useful for code that should NOT run on error. / 无异常时执行。
- `finally`: Runs **no matter what** —exception or not. Perfect for cleanup (closing files, releasing resources). / 无论如何都会执行（常用于清理资源）。

类似于 `if-else`，但专门处理异常。`try` 块出错时自动执行 `except` 块。

【Example / 实战案例】Specific Exception Handling / 精确捕获异常类型

永远避免裸 `except:`！始终指定具体的异常类型：

```python
# BAD — bare except catches EVERYTHING (even Ctrl+C!)
try:
    result = 10 / 0
except:                                 # Don't do this!
    print("Something went wrong")

# GOOD — catch specific exceptions
try:
    user_input = input("Enter a number: ")
    result = 10 / float(user_input)
except ValueError:                      # Catches bad float conversion
    print("That's not a valid number!")
except ZeroDivisionError:               # Catches division by zero
    print("Cannot divide by zero!")
```

【Example / 实战案例】else, finally, and raise / else · finally · raise 完整结构

```python
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Division by zero!")
        raise                           # Re-raise — let caller handle it
    except TypeError as e:              # Capture the exception object
        print(f"Type error: {e}")
        raise ValueError("Both arguments must be numbers") from e
    else:
        print(f"Success! Result = {result}")  # Only runs if no exception
        return result
    finally:
        print("Cleanup: this ALWAYS runs.")   # Runs no matter what!

# Test cases / 测试
safe_divide(10, 2)      # Success → result=5.0, cleanup runs
# safe_divide(10, 0)    # ZeroDivisionError → cleanup STILL runs!
# safe_divide(10, "x")  # TypeError → chained to ValueError

# Key points:
# - else: runs ONLY on success (avoids catching its own exceptions)
# - finally: ALWAYS runs — even after return/raise/exception
# - raise: re-throws the same exception to the caller
# - raise X from Y: chains exceptions for better debugging
```

6.5.3 Debugging Tips / 调试技巧

【Concept / 核心概念】Essential Debugging Techniques / 调试必备技巧

1. **print() debugging / 打印调试**: Insert `print()` calls at key points to track variable values and program flow. Simplest and most universal technique.
   在关键位置插入 `print()` 来追踪变量值和程序流程。

2. **Read the error message / 读错误信息**: Python's traceback tells you the **file**, **line number**, and **error type**. Read it from **bottom to top** —the last line is the actual error.
   Python 的错误回溯从下往上看：最后一行是真正的错误。

3. **Simplify the problem / 简化问题**: Comment out code until the error disappears, then add it back piece by piece. Isolate the minimal reproducible case.
   逐段注释代码直到错误消失，再逐段加回来，定位最小重现案例。

4. **Check your assumptions / 检查假设**: Use `type()` to verify variable types. Use `len()` to check data sizes. Print intermediate results.
   用 `type()` 检查变量类型，用 `len()` 确认数据长度。

5. **Rubber duck debugging / 小黄鸭调试法**: Explain your code line by line to an imaginary listener. The act of explaining often reveals the bug.
   逐行向" 小黄鸭" 解释你的代码，解释的过程往往就能发现问题。

【Example / 实战案例】Debugging Workflow Demo / 调试流程演示

```python
# Problem: function returns unexpected result
def calculate_average(grades):
    total = 0
    for g in grades:
        total += g
    return total / len(grades)

grades = [85, 92, 78]
result = calculate_average(grades)
print(f"Result: {result}")          # 85.0 — correct!

# But what if we have an empty list?
# calculate_average([])             # ZeroDivisionError — crash!

# Debugging approach:
# 1. Add print to check input
def calculate_average_v2(grades):
    print(f"DEBUG: grades = {grades}, type = {type(grades)}")  # Check input
    if len(grades) == 0:
        return 0                    # Guard clause / 防御性检查
    total = 0
    for g in grades:
        total += g
    result = total / len(grades)
    print(f"DEBUG: total={total}, count={len(grades)}, avg={result}")
    return result

```

```
print(calculate_average_v2([]))      # 0 — safe!
print(calculate_average_v2([1, 2, 3, 4, 5]))  # 3.0 — verified!
```

6.6 PEP-8 Style Guide / PEP-8 编程规范

【Concept / 核心概念】Professional Coding / 像专家一样写代码

Writing nice, easy-to-understand code is crucial. For Python, the official style guide is **PEP-8**.
写出易读、易懂的代码至关重要。Python 的官方代码风格指南是 PEP-8。
Conventions are specified in a strict manner, but remember: don't spend more time with the style guide than with your actual code.

6.6.1 Comments and Docstrings / 注释与文档字符串

【Warning / 避坑指南】DOs and DO NOTs for Comments / 注释的规范

DOs / 应该做的:

- Add explanatory comments before lines that are not entirely obvious.
- Use the same indentation as the code they describe.
- Add docstrings as the first line of every function and module.

DO NOTs / 不应该做的:

- Do NOT add self-explanatory comments: `x = x + 1 # increment x` is BAD.
- Do NOT add comments that are wrong —worse than no comment at all!

6.6.2 Whitespaces / 空格规范

【Concept / 核心概念】Where to Put Spaces / 空格的位置

DO put spaces / 应该加空格的:

- After commas, colons, and semicolons.
- Around `=`, `+=` in assignments.
- Around most binary operators, e.g., `2 + 2`.

DO NOT put spaces / 不应该加空格的:

- Between a function name and `()`: `func(x)` NOT `func (x)`.
- Around `=` in keyword arguments: `n=42` NOT `n = 42`.
- After `(` and before `)`.

【Example / 实战案例】Good vs Bad Formatting / 好代码 vs 坏代码

```
# Bad / 坏代码
def my_function ( a,b,c ) :
```

```
    some_dict={"a" : "b", 4:5}
    another_function(a+b+c, n = 42)

# Good / 好代码
def my_function(a, b, c):
    some_dict = {"a": "b", 4: 5}
    another_function(a + b + c, n=42)
```

6.6.3 Blank Lines / 空行规范

【Concept / 核心概念】How to Use Blank Lines / 如何使用空行

- Separate function definitions by **two** blank lines.
- Separate code blocks (loops, if-statements) by a **one** blank line.
- Separate logical segments in your code by line breaks.
- **Rule of thumb**: No more than 4 lines without a blank line.
- Break up long lines of code into multiple ones —either logically (intermediate variables) or syntactically (line continuation with `\`).

【Example / 实战案例】Line Continuation / 行续接

```
# Explicit continuation with backslash
total_income = gross_income \
               - taxes \
               - student_loan_interest
```

6.6.4 Naming Conventions / 命名约定

【Concept / 核心概念】Standard Naming Patterns / 标准命名模式

- **Variables and functions**: `lower_case_with_underscores` (snake_case) —e.g., `translation_dict`, `word_counter`.
- **Constants**: `UPPER_CASE_WITH_UNDERSCORES` —e.g., `MAX_SIZE`.
- **Classes**: `UpperCamelCase` —e.g., `HREmployee`.
- **Avoid built-in names**: Do NOT use `list`, `str`, `int` as variable names.

6.7 Quick Reference / 速查表

Operation / 操作	Syntax / 语法
打开文件 (读)	`open('f.txt', 'r')`
打开文件 (写)	`open('f.txt', 'w')`
打开文件 (追加)	`open('f.txt', 'a')`
写入内容	`f.write(str)`
读取全部	`f.read()`
读取一行	`f.readline()`
读取所有行	`f.readlines()`
逐行遍历	`for line in f:`
关闭文件	`f.close()`
CSV 读 (Dict)	`csv.DictReader(f)`
CSV 写	`csv.writer(f).writerow(row)`
捕获异常	`try: ... except: ...`
命名规范 (变量/函数)	`snake_case`
命名规范 (常量)	`UPPER_CASE`
命名规范 (类)	`CamelCase`

7 07 - Visualization (Matplotlib) / 数据可视化

7.1 Introduction to Matplotlib / Matplotlib 简介

【Concept / 核心概念】What is Matplotlib? / 什么是 Matplotlib ?

Matplotlib is one of the most popular packages for quickly creating two-dimensional figures. NumPy makes manipulating data simple, but for **human consumption**, you need to visualize your data.
Matplotlib 是最流行的二维数据可视化库之一。

【Example / 实战案例】The First Plot / 第一个绘图

```
import matplotlib.pyplot as plt
import numpy as np

my_list = [10, 20, 25, 35]
plt.plot(my_list)           # Simple line plot
# In Jupyter: the plot displays automatically
# In scripts: add plt.show() to display
```

7.2 Anatomy of a "Plot" / 绘图结构解剖

【Concept / 核心概念】The Container Hierarchy / 容器层级结构

Matplotlib organizes elements in a hierarchy of containers:

1. **Figure**: The top-level container —the overall window or page. A Figure can contain multiple Axes/Subplots.
2. **Axes / Subplots**: The actual plot area inside the figure where data is drawn. Each Axes has its own coordinate system.
3. **Axis**: Each Axes has an **XAxis** and a **YAxis**, which contain ticks, tick labels, and data limits.

简单类比：**Figure** 是画板，**Axes** 是画板上的坐标系（子图），**Axis** 是坐标轴。

7.3 Figure & Subplots Management / 画板与子图管理

7.3.1 Creation / 显式创建

【Example / 实战案例】Using plt.subplots() / 显式创建 Figure 和 Axes

To manage multiple plots, it makes sense to explicitly create `figure` and `axes` objects.

```
import matplotlib.pyplot as plt
import numpy as np

# Create a single plot — returns (figure, axes)
fig, ax = plt.subplots(nrows=1, ncols=1)

# Create a grid of subplots — 2 rows, 4 columns
fig, axes = plt.subplots(nrows=2, ncols=4)
# axes is now a 2x4 array of Axes objects
```

7.3.2 Appearance Optimization / 视觉优化

【Concept / 核心概念】fig.tight_layout() and figsize / 紧凑布局与画板尺寸

- `fig.tight_layout()`: Automatically adjusts margins so labels don't overlap.
- `figsize`: Sets the figure size in inches, e.g., `figsize=(10, 4)`.

【Example / 实战案例】Improving Appearance / 改善外观

```
# Without tight_layout — labels may overlap!
fig, axes = plt.subplots(nrows=2, ncols=4)
fig.tight_layout()                      # Fix overlapping

# With figsize — control the overall size
fig, axes = plt.subplots(nrows=2, ncols=4, figsize=(10, 4))
fig.tight_layout()
```

7.3.3 Setting Up Axes / 配置坐标轴

【Example / 实战案例】Using Explicit Setters / 显式配置方法

Matplotlib's objects have lots of explicit setters —functions that start with `set_<something>`:

```
fig, ax = plt.subplots(nrows=1, ncols=1)

ax.set_xlim([0.5, 4.5])             # Set x-axis range
ax.set_ylim([-2, 8])                # Set y-axis range
ax.set_title("An Example Axes")     # Set title
ax.set_ylabel("Y-Axis")             # Set y-axis label
ax.set_xlabel("X-Axis")             # Set x-axis label

```

```
fig                                    # In Jupyter: display the figure
```

7.4 Core Plotting Functions / 核心绘图函数

7.4.1 Conditional Bar Plots / 条件条形图

【Example / 实战案例】Dynamically Colored Bars / 动态着色条形图

```
np.random.seed(1)
x = np.arange(5)
y = np.random.randn(5)

fig, ax = plt.subplots()
vert_bars = ax.bar(x, y)

# Color negative bars red / 负值柱体标红
for bar, height in zip(vert_bars, y):
    if height < 0:
        bar.set(color='red')

ax.axhline(y=0, color='black', linewidth=2)  # Add zero line
```

7.4.2 Scatter for N-Dimensional Data / 多维散点图

【Concept / 核心概念】Mapping Multiple Dimensions / 多维度映射

Scatter allows mapping data to **x-position**, **y-position**, **color (c)**, and **size (s)** —enabling visualization of multiple dimensions simultaneously.
散点图可以将多个维度分别映射到 x 位置、y 位置、颜色和大小。

【Example / 实战案例】Multidimensional Scatter / 多维散点图

```
n = 50
x1 = np.random.random(n)
x2 = np.random.random(n) * 50
x3 = np.random.random(n)
y = x1 + x2 + x3

fig, ax = plt.subplots()
sc = ax.scatter(x=x1, y=y, s=x2, c=x3, marker='o')
fig.colorbar(sc)                # Add color scale bar
```

7.4.3 Image Display / 图像显示

【Example / 实战案例】Using imshow() / imshow() 显示图像

```
fig, ax = plt.subplots()
img = plt.imread("images/cat.jpg")
ax.imshow(img)
ax.axis("off")                    # Hide axis ticks and labels
```

7.5 The "Language" of Matplotlib / 绘图视觉语言

7.5.1 Colors / 颜色规范

【Concept / 核心概念】Five Ways to Specify Colors / 五种颜色指定方式

1. **Single letters**: `'r'` (red), `'b'` (blue), `'g'` (green), `'c'` (cyan), `'m'` (magenta), `'y'` (yellow), `'k'` (black), `'w'` (white).
2. **HTML/CSS color names**: e.g., `'burlywood'`, `'chartreuse'`, `'yellowgreen'` (147 available).
3. **Hex values**: e.g., `#0000FF` for blue, `#00ffff` for cyan.
4. **RGB[A] tuples**: Floats in range [0, 1], e.g., `(1.0, 0.0, 0.0, 1.0)` for red. The alpha channel controls transparency.
5. **Grayscale strings**: `'0.0'` (black) to `'1.0'` (white), e.g., `'0.75'` for light gray.
6. **Cycle references**: `'C0'` to `'C9'` reference the style's default color cycle.

【Example / 实战案例】Color Specification Examples / 颜色指定示例

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()

# Single letters
ax.plot(t, t, linewidth=5, color="r")

# Hex values
ax.plot(t, t**2, linewidth=5, color='#00ffff')

# RGB tuple (with alpha)
ax.plot(t, t**3, linewidth=5, color=(0, 0, 1, 0.5))

# Grayscale
ax.plot(t, -t, linewidth=5, color='0.5')
plt.show()
```

7.5.2 Markers / 标记符号

Marker	Description	Marker	Description	Marker	Description
`'.'`	point / 点	`'o'`	circle / 圆	`'s'`	square / 方形
`'*'`	star / 星	`'+'`	plus / 加号	`'x'`	cross / 叉
`'D'`	diamond / 钻石	`'p'`	pentagon / 五边形	`'v'`	triangle_down
`'<'`	triangle_left	`'>'`	triangle_right	`'^'`	triangle_up
`'None'`	nothing / 无标记	`""`	nothing / 无标记		

7.5.3 Linestyles / 线型

Linestyle	Description / 描述
`'-'`	solid / 实线
`'--'`	dashed / 虚线
`'-.'`	dash-dot / 点划线
`':'`	dotted / 点线
`'None'` / `''`	draw nothing / 不画线

【Warning / 避坑指南】Don't Mix Up "-" and "-." / 不要混淆"-" 和"-."

- `'.-'` = line with dot markers / 实线 + 点标记
- `'-.'` = dash-dot line / 点划线

这两个看起来很相似但含义完全不同!

【Example / 实战案例】Marker and Linestyle Examples / 标记与线型示例

```
t = np.arange(0.0, 5.0, 0.2)
fig, ax = plt.subplots()

# With explicit arguments, you can set marker and linestyle separately
ax.plot(t, t, linestyle='-', linewidth=5)
ax.plot(t, t**2, linestyle='--', linewidth=5)
ax.plot(t, t**3, linestyle='-.', linewidth=5)
ax.plot(t, -t, linestyle=':', linewidth=5)
plt.show()
```

7.5.4 Colormaps / 颜色映射

【Concept / 核心概念】jet vs viridis / 两种经典色图

A **colormap** relates a scalar value to a color. Historically, `'jet'` was the default, but it has perceptual issues. `'viridis'` is the new default since Matplotlib v2.0 —it's perceptually uniform and colorblind-friendly.
色图将数值映射为颜色。`'viridis'` 是现代推荐默认值，对色盲友好。

【Example / 实战案例】Colormap Comparison / 色图对比

```
example_x = np.random.normal(size=500)
example_y = np.random.normal(size=500)
example_z = example_x + example_y

plt.scatter(example_x, example_y, c=example_z, cmap="jet")
plt.show()

plt.scatter(example_x, example_y, c=example_z, cmap="viridis")
plt.show()
```

7.6 Advanced Decoration / 进阶装饰与细节

7.6.1 Mathtext (LaTeX in Plots) / 数学公式

【Concept / 核心概念】LaTeX-Style Math in Plots / 图中的 LaTeX 数学公式

Any text surrounded by dollar signs (`$`) will be treated as **mathtext**. Use `r` prefix for "raw" strings to avoid backslash escape issues.
用 `$` 包裹的文本会被渲染为数学公式。建议用 `r` 原始字符串前缀避免转义问题。

【Example / 实战案例】Mathtext Example / 数学公式示例

```
fig, ax = plt.subplots()
ax.scatter([1, 2, 3, 4], [4, 3, 2, 1])
ax.spines['top'].set(visible=False)          # Remove top border
ax.spines['right'].set(visible=False)        # Remove right border
ax.set_title(r'$\sigma_i=\frac{3}{5}$', fontsize=25)
plt.show()
```

7.6.2 Legends / 图例

【Concept / 核心概念】Adding Legends / 添加图例

Provide a `label` to your plot, then call `ax.legend()` to automatically build a legend.
给绘图添加 `label`，然后调用 `ax.legend()` 自动生成图例。

【Example / 实战案例】Legend Example / 图例示例

```python
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 30], label='Paris')
ax.plot([1, 2, 3, 4], [30, 23, 13, 4], label='Lyon')
ax.set(ylabel='Temperature (deg C)', xlabel='Time', title='A tale of two cities')
ax.legend()                          # Automatically builds legend from labels
# ax.legend(loc='center')            # Use loc to position the legend
plt.show()
```

【Concept / 核心概念】Legend Location Codes / 图例位置代码

The `loc` keyword argument positions the legend. `'best'` (default) automatically chooses the location that overlaps plot elements the least.

Location String	Description / 位置
`'best'`	Auto-select / 自动选择
`'upper right'` / `'upper left'`	Top corners / 上角
`'lower right'` / `'lower left'`	Bottom corners / 下角
`'center'`	Center / 居中
`'right'` / `'center left'` / `'center right'`	Side positions / 侧边
`'upper center'` / `'lower center'`	Center top/bottom / 上/下中

7.6.3 Ticks and Tick Labels / 刻度与刻度标签

【Concept / 核心概念】Terminology / 术语辨析（重要）

- **Tick**: The **location** of a tick label on the axis. / 刻度线位置。
- **Tick Line**: The line that denotes the location of the tick. / 刻度线本身。
- **Tick Label**: The **text** displayed at that tick. / 刻度标签文字。
- **Ticker**: Automatically determines ticks for an Axis and formats tick labels. / 自动确定刻度位置和格式的组件。

【Example / 实战案例】Manual Tick Configuration / 手动配置刻度

```python
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 25, 35])

# Manually set ticks and tick labels on the x-axis
ax.xaxis.set(ticks=range(1, 5), ticklabels=[3, 100, -12, "foo"])

# Customize tick appearance
ax.tick_params(axis='y', direction='in', length=10)
plt.show()
```

【Example / 实战案例】Tick Labels from Data / 从数据生成刻度标签

```
data = [('apples', 2), ('oranges', 3), ('peaches', 1)]
fruit, value = zip(*data)               # Unzip into two tuples

fig, ax = plt.subplots()
x = np.arange(len(fruit))
ax.bar(x, value, align='center', color='gray')
ax.set(xticks=x, xticklabels=fruit)
plt.show()
```

7.6.4 Spines / 边框控制

【Example / 实战案例】Removing Spines / 移除边框

```
ax.spines['top'].set(visible=False)     # Remove top border
ax.spines['right'].set(visible=False)   # Remove right border
# Also available: 'bottom', 'left'
```

7.6.5 Subplot Spacing / 子图间距

【Concept / 核心概念】fig.subplots_adjust() / 调整子图间距

Spacing between subplots can be adjusted. A common gotcha: labels are NOT automatically adjusted to avoid overlapping those of another subplot.
子图标签不会自动避免重叠——需要使用 `tight_layout()` 或手动调整。

【Example / 实战案例】Subplot Spacing / 子图间距调整

```
fig, axes = plt.subplots(2, 2, figsize=(9, 9))
fig.subplots_adjust(wspace=0.3, hspace=-0.7,
                    left=0.125, right=0.8,
                    top=0.7, bottom=0.2)
plt.show()
```

【Example / 实战案例】Tight Layout for Multiple Subplots / 多子图的紧凑布局

```
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(nrows=2, ncols=2)

# Without tight_layout — labels overlap
example_plot(ax1); example_plot(ax2)
example_plot(ax3); example_plot(ax4)
plt.show()

# With tight_layout — everything fits!
fig.tight_layout()
```

```
plt.show()
```

7.7 Quick Reference / 速查表

Operation / 操作	Syntax / 语法
导入	`import matplotlib.pyplot as plt`
创建单图	`fig, ax = plt.subplots()`
创建多子图	`fig, axes = plt.subplots(2, 4)`
折线图	`ax.plot(x, y)`
条形图	`ax.bar(x, y)`
散点图	`ax.scatter(x, y, s=size, c=color)`
显示图像	`ax.imshow(img)`
紧凑布局	`fig.tight_layout()`
设置标题	`ax.set_title(str)`
设置轴标签	`ax.set_xlabel(str)` / `ax.set_ylabel(str)`
添加图例	`ax.legend()`
颜色条	`fig.colorbar(sc)`
隐藏轴	`ax.axis("off")`
隐藏边框	`ax.spines['top'].set(visible=False)`

8 08 - Pandas Library I / Pandas 库 I

8.1 Introduction to Pandas / Pandas 简介

【Concept / 核心概念】What is Pandas? / 什么是 Pandas？

Pandas is one of the most important libraries of Python for data analysis. It is well suited for:

- Tabular data with heterogeneously-typed columns (like SQL tables or Excel spreadsheets).
- Ordered and unordered time series data.
- Arbitrary matrix data with row and column labels.
- Any other form of observational / statistical data sets.

Install: `pip install pandas`. Import: `import pandas as pd`.

8.2 The Core Structures / Pandas 两大核心结构

【Concept / 核心概念】Series vs DataFrame / 一维 Series vs 二维 DataFrame

- **Series**: The 1-dimensional analog of the DataFrame. Think of it as a single column with labels. It is size-immutable —operations that change its size are not allowed.
  Series 是一维结构，类似带标签的单列数据。每个 DataFrame 的列本质上都是一个 Series。
- **DataFrame**: A tabular representation of data (rows and columns), similar to a spreadsheet or SQL table. It is **size-mutable** —data can be added or deleted.
  DataFrame 是二维表格结构，其大小可动态调整（增删行列）。

【Concept / 核心概念】The Peculiarity of Index / 索引的特殊性

In Pandas, **Index** are about **labels**, not just performance (like in SQL). They help with:

1. **Selection**: Label-based data access.
2. **Automatic alignment**: When performing operations between two DataFrames or Series, Pandas automatically aligns data by index labels.

在 Pandas 中，索引是关于" 标签" 的。它帮助在两个表格运算时自动对齐数据。
Special Index types: `MultiIndex` (hierarchical), `DatetimeIndex` (datetimes), `Float64Index` (floats), `CategoricalIndex` (categories).

8.3 Creating DataFrames / 创建表格的多种方式

8.3.1 The DataFrame Constructor / DataFrame 构造函数

【Concept / 核心概念】Constructor Signature / 构造函数签名

`pd.DataFrame(data=None, index=None, columns=None, dtype=None, copy=False)`

- `data`: Input —dict, list, set, ndarray, iterable, or DataFrame.
- `index`: (Optional) Row index list. Default: range of integers 0, 1, ···, n.
- `columns`: (Optional) Column labels list. Default: range of integers 0, 1, ···, n.
- `dtype`: (Optional) Data type for the whole DataFrame. By default, inferred from data.

8.3.2 From Dict / 从字典创建

【Example / 实战案例】Creating DataFrame from Dict / 字典 → DataFrame

Dict keys become column labels; dict values become column data.

```
import pandas as pd

# Simple dict / 简单字典
student_dict = {'Name': ['Joe', 'Nat'], 'Age': [20, 21], 'Marks': [85.10, 77.80]}
student_df = pd.DataFrame(student_dict)
print(student_df)
#    Name  Age  Marks
# 0   Joe   20  85.10
# 1   Nat   21  77.80

# With custom indices / 自定义索引
df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],
                   'C': [0.496714, -0.138264, 0.647689]},
                  index=['i', 'ii', 'iii'])
print(df)
```

8.3.3 From List / 从列表创建

【Example / 实战案例】Creating DataFrame from List / 列表 → DataFrame

By default, all list elements are added as a **single row**.

```
# Simple list — becomes one row / 列表变单行
fruits_list = ['Apple', 10, 'Orange', 55.50]
fruits_df = pd.DataFrame(fruits_list)
print(fruits_df)

# With custom column names and index / 自定义列名和索引
fruits_list = ['Apple', 'Banana', 'Orange', 'Mango']
fruits_df = pd.DataFrame(fruits_list, columns=['Fruits'],
                         index=['Fruit1', 'Fruit2', 'Fruit3', 'Fruit4'])
```

```
print(fruits_df)

# Multi-dimensional list / 多维列表
fruits_list = [['Apple', 'Banana', 'Orange', 'Mango'], [120, 40, 80, 500]]
fruits_df = pd.DataFrame(fruits_list)
print(fruits_df)        # Each sublist becomes a row

# Transpose to swap rows/columns / 转置行列互换
fruits_df = pd.DataFrame(fruits_list).transpose()
print(fruits_df)
```

8.3.4 From CSV / 从 CSV 文件创建

【Example / 实战案例】Reading CSV Files / 读取 CSV 文件

`pd.read_csv("file.csv")` is the most common way to load large datasets.

```
data1 = pd.read_csv("data/sample.csv")
print(type(data1))              # <class 'pandas.core.frame.DataFrame'>
print(data1.iloc[:, 2])         # Select 3rd column by position
```

8.4 Indexing & Slicing / 索引与切片（期末核心考点）

【Warning / 避坑指南】Label-based (.loc) vs Position-based (.iloc)

这是 Pandas 初学者最容易混淆的地方，务必背诵以下区别：

- `.loc[row_labels, column_labels]`：Based on **Labels**. Slicing is **INCLUSIVE** of the right boundary!
  例：`df.loc['i':'iii']` 返回包含 `'iii'` 在内的所有行。
- `.iloc[row_positions, column_positions]`：Based on **Integer Positions**. Slicing is **EXCLUSIVE** of the right boundary (standard Python behavior).
  例：`df.iloc[0:2]` 只返回第 0 和第 1 行，**不含**第 2 行。

【Example / 实战案例】Indexing Examples / 索引实战

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [True, True, False],
                   'C': [0.496714, -0.138264, 0.647689]},
                  index=['i', 'ii', 'iii'])

# Column Selection / 列选择
print(df['A'])                      # Single column → returns Series
print(df[['A', 'C']])               # Multiple columns → returns DataFrame

# Row Selection with .loc (label-based)
print(df.loc[['i', 'iii']])         # Select rows 'i' and 'iii'
print(df.loc['i':'iii'])            # INCLUSIVE of 'iii'!

```

```python
# Row Selection with .iloc (position-based)
print(df.iloc[[0, 1]])              # Select rows 0 and 1
print(df.iloc[:2])                  # EXCLUSIVE of index 2

# Combined row + column selection / 行列同时选择
print(df.loc['ii', 'C'])            # Scalar value at row 'ii', column 'C'
print(df.loc['i':'ii', ['A', 'C']]) # Rows 'i' to 'ii', columns A and C

# Scalar access / 单值快速访问
print(df.at[0, 'Name'])             # Fast label-based single value
print(df.iat[0, 0])                 # Fast position-based single value
```

【Concept / 核心概念】Column Selection Shortcuts / 列选择快捷方式

- `df['A']` —Single column, returns **Series**.
- `df[['A', 'C']]` —Multiple columns, returns **DataFrame**.
- `df.A` —Attribute lookup (works only if column name has no spaces and doesn't conflict with DataFrame methods).

8.5 Series / Series 详解

【Example / 实战案例】Creating and Selecting Series / Series 的创建与选择

```python
# Three ways to select a column as Series / 三种选取列的方式
df['A']                  # __getitem__ style
df.loc[:, 'A']           # .loc style (all rows, column 'A')
df.A                     # Attribute lookup style

# Create a Series directly / 直接创建 Series
s = pd.Series([2, 4, 6, 8, 10])
print(s)
# 0    2
# 1    4
# 2    6
# 3    8
# 4    10
# dtype: int64
```

Series share many of the same methods as DataFrames.

8.6 Inspecting Metadata / 查看元数据与统计

【Concept / 核心概念】Key Inspection Functions / 核心检查函数

- `df.info()`: Gives index range, column list, non-null counts, data types, and memory usage.
- `df.describe()`: Summary statistics (count, mean, std, min, 25%, 50%, 75%, max) for **numerical columns only**.
- `df.shape`: Returns tuple `(rows, columns)`.

【Example / 实战案例】Metadata and Statistics / 元数据与统计示例

```
data1 = pd.read_csv("data/sample.csv")
data1.info()            # Full metadata summary
data1.describe()        # Statistics for numerical columns only
print(data1.shape)      # (rows, columns)
```

8.6.1 DataFrame Attributes / DataFrame 属性一览

Attribute / 属性	Description / 描述
`df.index`	Range of the row index / 行索引范围
`df.columns`	List of column labels / 列标签列表
`df.dtypes`	Column names and their data types / 列名及数据类型
`df.values`	All rows as numpy array / 所有数据转为 numpy 数组
`df.empty`	Check if DataFrame is empty / 检查是否为空
`df.size`	Total number of values (rows × columns) / 值的总数
`df.shape`	Number of rows and columns / 行列数元组

8.7 DataFrame Selection / 数据选择函数

【Example / 实战案例】head, tail, at, iat, get / 选择函数

```
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
                'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)

# Top N rows / 前 N 行
student_df.head(2)

# Bottom N rows / 后 N 行
student_df.tail(2)

# Single value by label / 按标签获取单值
```

```
student_df.at[0, 'Name']              # 'Joe'

# Single value by position / 按位置获取单值
student_df.iat[0, 0]                  # 'Joe'

# Get column / 获取列
student_df.get('Name')                # Returns Series
```

8.8 Data Manipulation / 数据修改操作

8.8.1 Inserting Columns / 插入列

【Example / 实战案例】df.insert() / 插入列

`df.insert(loc=col_position, column=new_col_name, value=default_value)`

```
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
                'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)

# Insert new column 'Class' at position 2 with default value 'A'
student_df.insert(loc=2, column="Class", value='A')
print(student_df)
```

8.8.2 Dropping Columns / 删除列

【Example / 实战案例】df.drop() / 删除列

`df.drop(columns=[col1, col2, ...])`

```
# Delete the 'Age' column
student_df = student_df.drop(columns='Age')
print(student_df)
```

8.8.3 Conditional Update with .where() / 条件替换

【Concept / 核心概念】.where() Logic / .where() 替换逻辑

`df.where(condition, other=new_value)`:

- If `condition` is **True** → keep the original value.
- If `condition` is **False** → replace with `other`.

条件为假则替换，条件为真则保持不变。

【Example / 实战案例】Conditional Replacement / 条件替换示例

```
student_dict = {'Name': ['Joe', 'Nat', 'Harry'], 'Age': [20, 21, 19],
                'Marks': [85.10, 77.80, 91.54]}
student_df = pd.DataFrame(student_dict)

# Replace marks < 80 with 0
filter = student_df['Marks'] > 80
student_df['Marks'].where(filter, other=0, inplace=True)
print(student_df)
# Joe: 85.10 stays (True), Nat: 77.80 → 0 (False), Harry: 91.54 stays (True)
```

8.8.4 Filtering Columns by Label / 按列标签过滤

【Warning / 避坑指南】.filter() filters LABELS, NOT data! / .filter() 过滤的是标签而非数据

`df.filter(like='N', axis='columns')` applies the filter to the **column labels or row index**, NOT the actual data inside the DataFrame.
`.filter()` 过滤的是" 列名/行名"，而不是数据内容!

【Example / 实战案例】.filter() Example / .filter() 示例

```
student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
                           'Age': [20, 21, 19],
                           'Marks': [85.10, 77.80, 91.54]})

# Only include columns whose label starts with 'N'
student_df = student_df.filter(like='N', axis='columns')
print(student_df)   # Only 'Name' column remains
```

8.8.5 Renaming Columns / 重命名列

【Example / 实战案例】df.rename() / 列重命名

`df.rename(columns={'old_name': 'new_name'})`

```
student_df = student_df.rename(columns={'Marks': 'Percentage'})
print(student_df)
```

8.9 Combining Data / 数据合并 (Join & Merge)

8.9.1 Join / 横向拼接

【Example / 实战案例】df.join() / 横向拼接

`df1.join(df2)` joins DataFrames by index.

```
student_df = pd.DataFrame({'Name': ['Joe', 'Nat'], 'Age': [20, 21]})
marks_df = pd.DataFrame({'Marks': [85.10, 77.80]})
joined_df = student_df.join(marks_df)
print(joined_df)
```

8.9.2 Merge / SQL 风格合并

【Concept / 核心概念】SQL-style Merging / SQL 风格合并

`pd.merge(df1, df2, how='...', on='column_name')`:

- `how='inner'`: Only keep rows where the key exists in **BOTH** tables. / 只保留两表都有的键。
- `how='left'`: Keep all rows from `df1`, fill missing `df2` data with `NaN`. / 保留左表所有行。
- `how='right'`: Keep all rows from `df2`, fill missing `df1` data with `NaN`. / 保留右表所有行。
- `how='outer'`: Keep all rows from **both** tables. / 保留两表所有行。

【Example / 实战案例】Merge Examples / 合并示例

```
df1 = pd.DataFrame({'Name': ['Joe', 'Nat', 'Davis'], 'Age': [20, 21, 23]})
df2 = pd.DataFrame({'Name': ['Joe', 'Nat', 'Alice'], 'Marks': [85.10, 77.80, 90.00]})

print(pd.merge(df1, df2, how='inner', on='Name'))    # Joe, Nat only
print(pd.merge(df1, df2, how='left', on='Name'))     # Joe, Nat, Davis
print(pd.merge(df1, df2, how='right', on='Name'))    # Joe, Nat, Alice
print(pd.merge(df1, df2, how='outer', on='Name'))    # Joe, Nat, Davis, Alice
```

8.10 Aggregation & Sorting / 分组聚合与排序

8.10.1 GroupBy / 分组聚合

【Concept / 核心概念】Split-Apply-Combine / 拆分-应用-组合

`df.groupby(col_label).mean()` splits data by groups, applies the aggregation, and combines results.

【Example / 实战案例】GroupBy Example / 分组聚合示例

```
student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
                           'Class': ['A', 'B', 'A'],
```

```
                            'Marks': [85.10, 77.80, 91.54]})

# Average marks per class / 每个班级的平均分
result = student_df.groupby('Class').mean()
print(result)
#          Marks
# Class
# A       88.32
# B       77.80

# Reset index to make 'Class' a column again
result = result.reset_index()
```

8.10.2 Sorting / 排序

【Example / 实战案例】df.sort_values() / 排序

`df.sort_values(by=['col1', 'col2'], ascending=[True, False])`

```
student_df = pd.DataFrame({'Name': ['Joe', 'Nat', 'Harry'],
                           'Age': [20, 21, 19],
                           'Marks': [85.10, 77.80, 91.54]})

# Sort by 'Marks' ascending
student_df = student_df.sort_values(by=['Marks'])
# Sort by 'Marks' descending
student_df = student_df.sort_values(by=['Marks'], ascending=False)
```

8.10.3 Iteration / 遍历

【Example / 实战案例】df.iterrows() / 逐行遍历

Returns `(index, row)` pairs for each row.

```
student_df = pd.DataFrame({'Name': ['Joe', 'Nat'], 'Age': [20, 21], 'Marks': [85, 77]})

for index, row in student_df.iterrows():
    print(index, row)
# 0 Name      Joe
#   Age        20
#   Marks      85
#   Name: 0, dtype: object
```

8.11 Conversion / 格式转换

【Example / 实战案例】df.to_dict() / 转回字典

```python
dict_result = student_df.to_dict()
print(dict_result)
# {'Name': {0: 'Joe', 1: 'Nat'}, 'Age': {0: 20, 1: 21}, ...}
```

8.12 Quick Reference / 速查表

Operation / 操作	Syntax / 语法
创建 DataFrame(字典)	`pd.DataFrame(dict)`
创建 DataFrame(列表)	`pd.DataFrame(list)`
读取 CSV	`pd.read_csv('file.csv')`
选择列	`df['col']` / `df[['c1','c2']]`
标签索引	`df.loc[row, col]`
位置索引	`df.iloc[row, col]`
单值 (标签)	`df.at[row, col]`
单值 (位置)	`df.iat[row, col]`
元数据	`df.info()`
统计摘要	`df.describe()`
形状	`df.shape`
前 N 行	`df.head(n)`
后 N 行	`df.tail(n)`
插入列	`df.insert(loc, name, value)`
删除列	`df.drop(columns=[...])`
条件替换	`df.where(cond, other=x)`
重命名	`df.rename(columns={old:new})`
合并 (SQL)	`pd.merge(df1, df2, how='inner', on='key')`
横向拼接	`df1.join(df2)`
分组聚合	`df.groupby('col').mean()`
排序	`df.sort_values('col')`
遍历	`df.iterrows()`
转字典	`df.to_dict()`

9 09 - Pandas Library II / Pandas 库 II

9.1 Setup & Data Loading / 环境配置与数据载入

【Example / 实战案例】Initial Setup / 初始化配置

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Pandas display options / 显示选项
pd.set_option('display.max_columns', 15)
pd.set_option('display.max_rows', 15)
pd.set_option('display.float_format', lambda x: '%.3f' % x)
```

9.1.1 Loading Data / 数据载入

【Concept / 核心概念】Universal Readers / 万能读取器

Pandas supports various formats:

- `pd.read_csv()`: Most common —loads comma-separated data.
- `pd.read_excel()`: Loads data from Excel sheets.
- `pd.read_html()`: Scrapes tables directly from a website URL.
- `pd.read_json()`: Read from JSON formatted string, URL or file.
- `pd.read_table()`: From a delimited text file (like TSV).

【Example / 实战案例】Reading the house.csv Dataset / 读取房屋数据集

```python
data = pd.read_csv('data/house.csv')
print(data.shape)       # (rows, columns) — check dataset size
data.head(5)            # View first 5 rows
data.tail(5)            # View last 5 rows
```

9.2 Data Cleaning / 数据清洗（实战灵魂）

9.2.1 Dropping Redundant Columns / 删除冗余列

【Concept / 核心概念】Why Drop Columns? / 为什么要删列？

Columns that have the same value for all rows (like 'country' or 'postcode') provide **no information** —they should be dropped.
对所有行都相同的列（如国家、邮编）不提供任何有效信息，应该删除。

【Example / 实战案例】Dropping Columns / 删除列

```
print(data.columns)                          # See all column names

# Drop columns that won't be used
data = data.drop(columns=["country", "postcode"])
print(data.shape)                            # Reduced column count
data.head(5)
```

9.2.2 Handling NaNs / 处理缺失值

【Warning / 避坑指南】NaN Detection and Deletion / 缺失值检测与删除

- `data.isna().sum()`: Counts missing values in each column. `isnull()` is equivalent.
- `data.dropna(inplace=True)`: Removes **any row** containing a missing value. `inplace=True` modifies the original DataFrame.
- `df.fillna(value)`: Replaces all null values with a specific value (e.g., the mean, or 0).
- **axis variants**: `dropna(axis=1)` drops columns with nulls; `dropna(thresh=n)` keeps rows with at least n non-null values.

【Example / 实战案例】NaN Handling Example / 缺失值处理示例

```
# Check for NaNs / 检查缺失值
print(data.isna().sum())
print(data.isnull().sum())                   # Same as isna()

# Remove rows with any NaN / 删除含 NaN 的行
data.dropna(inplace=True)
print(data.shape)                            # Fewer rows after dropping

# Alternative: fill NaNs with mean / 替代方案：用均值填充
# data['column'].fillna(data['column'].mean(), inplace=True)
```

9.2.3 Renaming Columns / 重命名列

【Example / 实战案例】Renaming for Convenience / 简化列名

Long column names are hard to type repeatedly. Rename them for convenience.

```
data.rename(columns={
    'rentEstimate_currentPrice': 'rentPrice',
    'saleEstimate_currentPrice': 'salePrice'
}, inplace=True)
data.head(5)
```

9.2.4 Type Conversion / 类型转换

【Concept / 核心概念】Optimizing Data Types / 优化数据类型

Some data types (like `object`) should be `category` for memory efficiency.
将只有几个不同值的字符串列转为 `category` 类型可以大幅节省内存。

【Example / 实战案例】Type Conversion Example / 类型转换示例

```
print(data.dtypes)                      # Check current types

# Convert 'propertyType' from object to category
data['propertyType'] = data.propertyType.astype('category')
data.head(5)
```

9.3 Feature Engineering / 特征工程（进阶技巧）

9.3.1 lambda Functions / 匿名函数

【Concept / 核心概念】What is a lambda? / 什么是 lambda？

Lambda functions are small, **one-line**, **anonymous** functions. They can receive multiple arguments but can only contain **one expression** (the return value).
Lambda 是小型的一行匿名函数，只能包含一个表达式。

```
# Syntax: lambda arguments: expression
square = lambda x: x**2
print(square(5))    # 25
```

9.3.2 Creating New Columns / 创建新列

【Example / 实战案例】Boolean Flag Column / 布尔标记列

```
# Create a boolean column 'above_mean'
data['above_mean'] = data.salePrice > data.salePrice.mean()
data.head(5)
```

【Example / 实战案例】Using assign() / 使用 assign() 方法

`assign()` also creates new columns using lambda:

```
data.assign(above_mean=lambda x: x.salePrice > x.salePrice.mean())
data.head(5)
```

9.3.3 Filtering Data / 数据过滤

【Example / 实战案例】Multi-Condition Filtering / 多条件过滤

Use `&` (and), `|` (or) between conditions. Each condition must be wrapped in **parentheses**!

```
# Filter: flat type, bedrooms >= 2, bathrooms <= 2
filtered = data.loc[(data['propertyType'] == 'Purpose Built Flat') &
                    (data['bedrooms'] >= 2.0) &
                    (data['bathrooms'] <= 2.0)]
print(filtered)
```

【Warning / 避坑指南】Use & not 'and' for Element-Wise Logic / Pandas 中要用 & 而不是 and

`and`/`or` work on single booleans. `&`/`|` work element-wise on Series/arrays. Every condition MUST be wrapped in parentheses: `(cond1) & (cond2)`.

9.3.4 Conditional Modification / 条件修改

【Example / 实战案例】Modifying Values Based on Condition / 条件修改值

```
# Check value distribution / 查看值分布
print(data['above_mean'].value_counts())

# Swap True/False labels / 交换标签
data.loc[data['above_mean'] == False, 'above_mean'] = 'yes'
data.loc[data['above_mean'] == True, 'above_mean'] = 'no'
data.head(5)
```

9.3.5 Sorting by Multiple Columns / 多列排序

【Example / 实战案例】Multi-Column Sort / 多列排序

```
# Sort salePrice ascending, bathrooms descending
data.sort_values(['salePrice', 'bathrooms'], ascending=[True, False])
# Equivalent: ascending=[1, 0] where 1=ascending, 0=descending
```

9.3.6 Discretization (Binning) / 数据离散化（分箱）

【Concept / 核心概念】pd.cut() / 分箱操作

Converts a continuous numerical variable into discrete "bins" (categories).
将连续数值变量划分为离散的"箱子"(分类)。`pd.cut(data, bins=3, labels=['low', 'medium', 'high'])`

【Example / 实战案例】Price Binning Example / 价格分箱示例

```
# Divide salePrice into 3 equal-width bins
data["salePriceBins"] = pd.cut(data["salePrice"], bins=3,
                               labels=['low', 'medium', 'high'])
data.head(5)
print(data["salePriceBins"].value_counts())  # Count per bin
```

9.3.7 Encodings / 类别数据编码

【Concept / 核心概念】Two Encoding Methods / 两种编码方式

1. **LabelEncoder**: Converts strings into integers (e.g., 'Freehold' → 0, 'Leasehold' → 1). Use when there is an **ordinal** relationship.
2. **One Hot Encoding (`pd.get_dummies`)**: Creates N new binary columns for N categories. Each column contains 0 or 1. Use when categories are **nominal** (no order).

【Example / 实战案例】LabelEncoder Example / 标签编码

```
from sklearn.preprocessing import LabelEncoder

# Check current values / 查看当前值
print(data.tenure.value_counts())

# Encode: Freehold → 0, Leasehold → 1
tenure_encoder = LabelEncoder()
data['tenure'] = tenure_encoder.fit_transform(data['tenure'])
print(data.head(5))
print(data.tenure.value_counts())

# See the mapping / 查看映射关系
print({k: v for k, v in enumerate(list(tenure_encoder.classes_))})
```

【Example / 实战案例】One Hot Encoding Example / 独热编码

```
# Create dummy columns for 'tenure'
data_encoder = pd.get_dummies(data, columns=["tenure"])
print(data_encoder.shape)           # Now has more columns
data_encoder.head()
```

9.4 Data Exploration & Aggregation / 数据探索与聚合

9.4.1 Group Analysis / 分组分析

【Example / 实战案例】Exploring by Bathrooms / 按浴室数量分析

```
print(data["bathrooms"].unique())              # All unique bathroom counts
print(data["bathrooms"].value_counts())         # Frequency of each count

print(data.groupby("bathrooms").size())         # Same as value_counts
print(data.groupby("bathrooms")['salePrice'].mean())  # Avg price per bathroom count
```

9.4.2 Pivot Table / 透视表

【Concept / 核心概念】Multi-dimensional Aggregation / 多维汇总

A **pivot table** is a table of grouped values that aggregates individual items into discrete categories.
透视表将数据按照两个维度进行分组并聚合。

【Example / 实战案例】Pivot Table Example / 透视表示例

```
# Mean salePrice by bathrooms AND price bins
data.pivot_table(index='bathrooms',
                 columns='salePriceBins',
                 values='salePrice',
                 aggfunc='mean')
```

This creates a 2D table: rows = bathroom count, columns = price bin, values = mean sale price.

9.4.3 Crosstabs / 交叉表

【Concept / 核心概念】Frequency Table / 频率表

`pd.crosstab()` provides an easy way to create a frequency table —counting how many occurrences exist for combinations of categories.
交叉表用于统计两个分类变量的组合出现频率。

【Example / 实战案例】Crosstab Example / 交叉表示例

```
pd.crosstab(index=data["bathrooms"], columns=data["salePriceBins"])
```

This shows: for each bathroom count, how many properties fall into each price bin.

9.4.4 Describe / 统计摘要

【Example / 实战案例】General Statistics / 整体统计

```
data.describe()
# Returns count, mean, std, min, 25%, 50%, 75%, max for all numerical columns
```

9.5 Visualization with Pandas / 数据可视化

【Concept / 核心概念】Integrated Plotting / 集成绘图

Pandas provides a `.plot()` method that directly uses Matplotlib. For SVG-format plots (high resolution), use:

```
%config InlineBackend.figure_formats = ['svg']
%matplotlib inline
```

9.5.1 Line Plot / 折线图

【Example / 实战案例】Line Plot with Pandas / Pandas 折线图

```
# Plot salePrice against rentPrice for rows 100-110
data.loc[100:110].plot(x="rentPrice", y="salePrice",
                       title='salePrice vs rentPrice',
                       ylabel='salePrice', alpha=0.8)
plt.show()
```

9.5.2 Bar Plot / 条形图

【Example / 实战案例】Customized Bar Plot from Pivot Table / 透视表条形图

```
# Create pivot table for plotting
plot_data = data.pivot_table(index='bathrooms', columns='salePriceBins',
                             values='salePrice', aggfunc='mean')

# Plot as bar chart with custom formatting
ax = plot_data.plot(
    kind='bar', rot=0, xlabel='', ylabel='Price Mean', fontsize=8,
    figsize=(12, 1.5), title='Number of bathrooms | Price Bins',
)
ax.legend(title='', loc='center', bbox_to_anchor=(0.5, -0.3),
          ncol=3, frameon=False)
plt.show()
```

A Complete Command Reference / 全课程命令速查表

Below is a consolidated quick reference of all key syntax, functions, and methods covered in Chapters 01–09. Use for exam review.

01. Variables, Data Types & Operators

Syntax	Description
`x = 10`	Assignment; value on RHS bound to name on LHS
`type(x)`	Return the data type of `x`
`print(x)`	Output `x` to console
`# comment`	Single-line comment, ignored by Python
`None`	Null value (`NoneType`); not equal to 0, False, or `""`
`int(x) / float(x) / str(x) / bool(x)`	Type casting; `int(9.9) → 9` truncates (does not round)
`bool(0) → False`	Truthiness: 0, 0.0, `""`, `None`, `[]`, `{}` are falsy
`input(prompt)`	Read user input; **always returns str**
`// / ** / %`	Floor division / exponent / modulo
`== != < > <= >=`	Comparison operators; result is always `bool`
`and / or / not`	Logical operators

Strings

Syntax	Description
`len(s)`	String length (includes spaces)
`s[i] / s[-i]`	Positive index (0-based) / Negative index (-1 = last char)
`s[start:stop]`	Slice; **inclusive:exclusive** (left-closed, right-open)
`s.lower() / s.upper()`	Convert to lower/upper case
`s.strip() / .lstrip() / .rstrip()`	Remove leading/trailing whitespace
`s.count(sub) / s.find(sub)`	Count occurrences / Find first index (-1 if not found)
`s.replace(old, new)`	Replace all occurrences; returns new string
`s.startswith(p) / s.endswith(p)`	Check prefix/suffix; returns `bool`
`s.split(d) / d.join(list)`	Split by delimiter / Join list with delimiter
`f"{var}"`	**f-string** (Python 3.6+); no `str()` needed
`"{} {}".format(a,b)`	`.format()` method
`"%s" % v`	Old-style `%` formatting (legacy)
Escape chars: `\n`, `\t`, `\\`, `\"`	Newline, tab, backslash, double-quote
Immutability	Strings are **immutable**; methods return new strings

02. Control Structures

Syntax	Description
`if cond: ... elif cond: ... else: ...`	Conditional branching
`for item in iterable: ...`	Iterate over each element in iterable
`for i in range(start, stop, step):`	Integer sequence loop; **inclusive:exclusive**
`while cond: ...`	Repeat while condition is True
`break`	Exit the innermost loop immediately
`continue`	Skip remaining code, jump to next iteration
`pass`	Placeholder; does nothing
`for i, val in enumerate(lst):`	Loop with both index and value
`for a, b in zip(l1, l2):`	Parallel iteration over multiple iterables
`[expr for x in lst if cond]`	**List comprehension** — one-line list generation

03. Data Structures

Syntax	Description
List / 列表	
`lst.append(x) / lst.extend(iter)`	Append element / Extend with all elements from iterable
`lst.insert(i, x) / lst.remove(x)`	Insert at index / Remove first match (error if missing)
`lst.pop(i) / lst.clear()`	Pop at index (default last) / Clear all elements
`lst.index(x) / lst.count(x)`	Find index of first match / Count occurrences
`lst.sort() / sorted(lst)`	In-place sort / Return new sorted list
`lst.reverse() / lst.copy()`	In-place reverse / Shallow copy
`del lst[i] / lst[i:j] = [...]`	Delete by index / Slice assignment
Tuple / 元组 — Immutable, hashable	
`tup = (1, 2, 3) / tup = 1, 2, 3`	Create tuple (parentheses optional)
`a, b = tup`	Tuple unpacking
Dictionary / 字典	
`d = {key: val}`	Create dictionary
`d[key] / d.get(key, default)`	Access (error if key missing) / Safe access with fallback
`d.keys() / .values() / .items()`	Views of keys / values / key-value pairs
`d.update({k:v}) / d.pop(key) / del d[key]`	Merge dict / Pop key / Delete key
`{k:v for k,v in d.items() if cond}`	Dict comprehension
Set / 集合 — Unordered, unique elements	
`s = {1,2,3} / set(lst)`	Create set (auto-deduplicates)
`s.add(x) / s.remove(x) / s.discard(x)`	Add / Remove (error if missing) / Safe remove
`s | t / s & t / s - t`	Union / Intersection / Difference

04. Functions & Modules

Syntax	Description
`def f(a, b=default): ... return x`	Function definition with optional default argument
`lambda x, y: x + y`	Anonymous function; single expression only
`import numpy / import numpy as np`	Import entire module / Import with alias
`from math import sqrt, pi`	Import specific names from module
`from module import *`	Import all public names (discouraged)
`dir(obj) / help(func)`	List attributes and methods / View docstring
`if __name__ == "__main__":`	Script entry-point guard: runs only when executed directly
`pip install package`	Install third-party package via terminal

05. Object-Oriented Programming

Syntax	Description
`class Dog:`	Class definition
`def __init__(self, name):`	Constructor; `self` refers to the instance
`self.name = name`	Instance attribute; unique to each object
`d = Dog("Buddy")`	Instantiation (automatically calls `__init__`)
`class Puppy(Dog):`	Inheritance: Puppy inherits all attributes/methods from Dog
`super().__init__(...)`	Call parent constructor from child class
`super().method()`	Call any parent method
`@property`	Decorator: turn method into attribute-like access (getter)
`@staticmethod`	Static method; no `self` parameter needed
`@classmethod`	Class method; first parameter is `cls`, not `self`
`__str__(self)`	String representation; called by `print(obj)` and `str(obj)`
`__repr__(self)`	Developer representation; called by `repr(obj)`

06. File I/O & Error Handling

Syntax	Description
`open('f.txt', 'r')`	Read mode; file must exist
`open('f.txt', 'w')`	Write mode; **overwrites existing content**!
`open('f.txt', 'a')`	Append mode; adds content at end of file
`with open('f.txt') as f:`	**Recommended**: context manager auto-closes file
`f.write(str)`	Write string to file; **does NOT add newline automatically**
`f.read()`	Read entire file as a single string
`f.readline()`	Read one line (includes trailing newline)
`f.readlines()`	Read all lines into a list of strings
`for line in f:`	Iterate line by line; **most memory-efficient**
`f.close()`	Manually close file (unnecessary with `with`)
`csv.reader(f) / csv.DictReader(f)`	CSV reader returning lists / dicts (**DictReader recommended**)
`csv.writer(f).writerow(row)`	Write one row to CSV file
`try: ... except ErrType: ... else: ... finally:`	Full exception handling structure
`SyntaxError / IndentationError`	Invalid Python syntax / Wrong indentation
`NameError / TypeError / ValueError`	Undefined variable / Wrong type operation / Invalid value
`IndexError / KeyError`	List index out of range / Dictionary key not found
`AttributeError / FileNotFoundError`	Missing attribute/method / File does not exist
`ZeroDivisionError`	Division by zero

PEP-8 Quick Reference

Rule	Convention
Variables and functions	`snake_case`: lowercase with underscores
Constants	`UPPER_CASE_WITH_UNDERSCORES`
Classes	`UpperCamelCase` (PascalCase)
Blank lines	2 blank lines between functions; 1 between code blocks
Operators	Spaces around operators: `x = a + b`
Commas	Space after comma: `f(a, b, c)`
Keyword arguments	No space around `=`: `f(n=42)`
Function call	No space before `(`: `func(x)`
Line length	Max 79 characters; break long lines with `\` or parentheses
Naming conflicts	Never use `list`, `str`, `int`, `print` as variable names

07. Matplotlib Visualization

Syntax	Description
`%matplotlib inline`	Enable inline display in Jupyter Notebook
`import matplotlib.pyplot as plt`	Import matplotlib's pyplot interface
`fig, ax = plt.subplots()`	Create figure and axes objects (**recommended style**)
`fig, axes = plt.subplots(nrows, ncols)`	Create a grid of subplots
`ax.plot(x, y)`	Line plot
`ax.scatter(x, y)`	Scatter plot
`ax.bar(x, height)`	Vertical bar chart
`ax.hist(data, bins)`	Histogram
`ax.boxplot(data)`	Box plot
`ax.set_title('title')`	Set chart title
`ax.set_xlabel('label') / set_ylabel('label')`	Set axis labels
`ax.set_xlim(lo, hi) / set_ylim(lo, hi)`	Set axis range limits
`ax.legend(loc=, frameon=)`	Display legend; `loc='best'` for automatic placement
`ax.grid(True)`	Add background grid
`ax.axis('off')`	Hide all axis elements
`plt.tight_layout()`	Auto-adjust spacing between subplots
`plt.savefig('f.png', dpi=300)`	Save current figure to file
`plt.show()`	Display the figure
Color specification	`'r','b','g','k','w'` letters / hex `#0000FF` / RGB `(0,0,1,0.5)` / grayscale `'0.5'` / cycle `'C0'--'C9'`
Markers	`'o'` circle, `'s'` square, `'D'` diamond, `'^'` triangle, `'*'` star
Linestyles	`'-'` solid, `'--'` dashed, `':'` dotted, `'-.'` dash-dot

08. Pandas I — Data Manipulation

Syntax	Description
`import pandas as pd`	Import Pandas library
`pd.read_csv('f.csv')`	Read CSV file into DataFrame
`pd.read_excel('f.xlsx')`	Read Excel file into DataFrame
`pd.DataFrame(dict)`	Create DataFrame from dictionary
`df.head(n) / df.tail(n)`	View first/last n rows (default: 5)
`df.shape`	Return (rows, columns) tuple
`df.columns / df.index`	Column names / Row index
`df.dtypes / df.info()`	Data types of each column / DataFrame info summary
`df.describe()`	Statistics: count, mean, std, min, 25%, 50%, 75%, max
`df.isna().sum()`	Count missing values per column; `isnull()` is equivalent
`df.dropna(inplace=True)`	Drop rows containing any NaN
`df.fillna(value)`	Replace all NaN values with `value`
`df.drop(columns=['c'])`	Drop specified columns
`df.rename(columns={old:new})`	Rename columns
`df.astype('category')`	Convert column type (saves memory for low-cardinality data)
`df.sort_values('c', ascending=False)`	Sort by column; can pass list for multi-column sort
`df['c'].unique()`	Return array of unique values
`df['c'].value_counts()`	Return frequency of each unique value
`df.loc[cond, cols]`	**Label-based indexing**: select by row label and column name
`df.iloc[i, j]`	**Integer-based indexing**: select by row/column position
`df.at[label, 'col'] / df.iat[i, j]`	Fast access to single scalar value
`df[df['col'] > 100]`	Boolean filtering: select rows meeting condition

09. Pandas II — Feature Engineering & Aggregation

Syntax	Description
`df.assign(new=lambda x: expr)`	Create new column using lambda expression
`df.groupby('col').mean()`	Group by column, compute mean (also sum, min, max, count, std)
`df.groupby('col').agg(['mean','std'])`	Apply multiple aggregation functions at once
`df.groupby(['c1','c2']).size()`	Group by multiple columns, count group sizes
`df.pivot_table(index, columns, values, aggfunc)`	**Pivot table**: multi-dimensional aggregation
`pd.crosstab(index, columns)`	**Crosstab**: frequency table of two categorical variables
`pd.cut(data, bins, labels)`	**Binning**: discretize continuous values into categories
`pd.qcut(data, q)`	Quantile-based binning
`pd.get_dummies(df, columns=['c'])`	**One-hot encoding**: create binary indicator columns
`LabelEncoder().fit_transform(series)`	**Label encoding**: convert strings to integers (sklearn)
`pd.merge(a, b, on='key', how='left')`	SQL-style merge; how = left/right/inner/outer
`pd.concat([df1, df2], axis=0)`	Concatenate DataFrames along rows (axis=0) or columns (axis=1)
`df.plot(kind='bar', rot, figsize, title)`	Pandas integrated plotting (calls Matplotlib internally)
`pd.set_option('display.max_columns', n)`	Configure Pandas display options